TUGAS AKHIR
DATABASE UNTUK SAINS DATA

"Optimalisasi Basis Data *Mobile Banking* untuk Meningkatkan Efisiensi dan Keamanan Transaksi di Suatu Bank"



KELOMPOK 6

Ferdinand Sitompul	(2206051576)
Haikal Fikri Rabani	(2206823713)
Muhammad Hilmy Putra R.	(2006534530)
Ester Oktafia	(2206810830)
Kartika Rizkia Zuhrah	(2206027993)

DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS INDONESIA

2024

DAFTAR ISI

DAFTAR ISI **2**
BAB I
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan 3
BAB II
RANCANGAN BASIS DATA **5**
2.1 Rancangan Database 5
2.2 ER (Entity Relationship) Diagram 10
2.3 Tabel Relasional 15
2.3.1 Tabel Relasional 15
2.3.2 Set Data 16
2.3.3 Normalisasi Data 17
BAB III
IMPLEMENTASI DAN PEMBAHASAN **20**
3.1 Implementasi SQL dan Testing 25
3.2 Implementasi Interaktif (GUI) dan Testing 31
BAB IV
PENUTUP **46**
4.1 Kesimpulan 46
4.2 Saran 46
DAFTAR PUSTAKA **47**
LAMPIRAN **48**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Layanan Mobile Banking telah menjadi solusi untuk mengakses dan melakukan transaksi finansial kapanpun dan dimanapun melalui perangkat mobile. Hal ini memberikan kemudahan yang signifikan bagi nasabah dalam mengelola keuangan mereka sehari-hari tanpa harus terbatas oleh lokasi dan waktu operasional bank. Namun, perlunya keamanan transaksi tetap menjadi hal yang sangat penting dan harus diperhatikan dengan serius.

Salah satu aspek yang esensial dalam menjaga keamanan transaksi Mobile Banking adalah transparansi informasi terkait dengan setiap transaksi yang dilakukan. Nasabah perlu memiliki akses yang jelas terhadap detail transaksi mereka, termasuk tanggal, waktu, dan jumlah transaksi yang dilakukan. Dengan demikian, nasabah dapat memverifikasi dan memantau aktivitas finansial mereka secara real-time, sehingga memungkinkan mereka untuk mendeteksi adanya kegiatan yang mencurigakan dengan cepat. Oleh karena itu, penyedia layanan Mobile Banking perlu memastikan bahwa platform mereka menyediakan informasi yang terperinci mengenai setiap transaksi yang dilakukan oleh nasabah.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dari *mini project* ini adalah sebagai berikut.

a. Bagaimana sistem basis data dapat memastikan ketersediaan data yang cepat dalam mendukung akses informasi pelanggan dan transaksi keuangan di bank?
b. Bagaimana sistem basis data mampu mengelola jutaan transaksi yang dilakukan nasabah dengan cepat dan akurat, serta mampu memastikan konsistensi data?
c. Bagaimana sistem basis data mampu menyediakan alat dan mekanisme untuk menganalisis data dengan cepat dan membuat laporan yang informatif?

1.3 Tujuan

Berdasarkan latar belakang dan rumusan masalah di atas, tujuan dari *mini project* ini adalah sebagai berikut.

a. Bank membutuhkan akses cepat dan andal terhadap informasi pelanggan dan transaksi keuangan. Sistem basis data dirancang untuk memastikan ketersediaan data yang tinggi dan waktu respons yang cepat.

b. Bank melakukan jutaan transaksi setiap hari, termasuk penyetoran, penarikan, transfer dana, dan pembayaran. Sistem basis data harus mampu mengelola transaksi ini dengan cepat dan akurat, serta memastikan konsistensi data dalam situasi transaksi yang kompleks.
c. Bank menggunakan data untuk menganalisis kinerja keuangan, perilaku pelanggan, risiko kredit, dan aspek lain dari operasi mereka. Sistem basis data menyediakan alat dan mekanisme untuk menganalisis data dengan cepat dan membuat laporan yang informatif.

BAB II
RANCANGAN BASIS DATA

2.1 Rancangan *Database*

Dalam membuat sebuah *database*, diperlukan rancangan *database* yang baik untuk memastikan data dapat disimpan dan dikelola dengan efisien. Rancangan ini mencakup penentuan entitas, atribut, dan hubungan antar entitas yang relevan dengan kebutuhan sistem. Berikut beberapa komponen dalam *database* yang perlu diperhatikan.

1. *Entity* (Entitas)

   Entitas merupakan suatu objek yang ada di dunia nyata, sehingga dapat dibedakan dari semua objek yang lain. Terdapat dua tipe entitas, yaitu *strong entity type* dan *weak entity type.* Entitas yang keberadaannya tidak bergantung pada entitas lain disebut sebagai *strong entity type* yang kerap kali disebut *parent, owner dominant.* Sedangkan, *weak entity type* adalah entitas yang keberadaannya bergantung pada entitas lain atau disebut *child dependent, subordinate.* Umumnya, pada ER diagram, entitas disimbolkan persegi panjang.

2. *Relationship* (Relasi)

   Relasi merupakan hubungan di antara beberapa entitas. Dalam suatu hubungan atau relasi entitas, terdapat kardinalitas relasi yang menunjukkan jumlah entitas yang dapat dihubungkan ke satu entitas lainnya dengan suatu *relationship sets*. Umumnya, pada ER diagram, relasi disimbolkan dengan jajar genjang. Jenis-jenis kardinalitas atau derajat relasi, meliputi:

   a. *One-to-One* (1-1), artinya relasi tersebut ada ketika untuk setiap satu entitas A akan berelasi dengan hanya dengan satu entitas B.
   b. *One-to-Many* (1-n), artinya relasi tersebut ada ketika untuk setiap satu entitas A akan berelasi lebih dari satu entitas B.
   c. *Many-to-One* (m-1), artinya relasi tersebut ada ketika untuk lebih dari satu entitas A akan berelasi dengan hanya dengan satu entitas B.
   d. *Many-to-Many* (m-n). artinya relasi tersebut ada ketika untuk lebih dari satu entitas A akan berelasi dengan lebih dari satu entitas B.

3. Atribut

   Atribut merupakan karakteristik dari entitas atau relasi yang menyediakan penjelasan rinci tentang entitas atau relasi itu sendiri. Nilai pada atribut merupakan data aktual atau informasi yang disimpan pada suatu atribut yang ada di dalam entitas

atau relasi. Untuk setiap atribut, memiliki domain *(value set)* atau batas-batas nilai yang diperbolehkan bagi suatu atribut. Umumnya, pada ER diagram, atribut disimbolkan dengan lingkaran. Terdapat beberapa jenis atribut yang digunakan dalam ER diagram, yaitu:

a. *Key*, yaitu atribut yang digunakan untuk menentukan suatu *entity* secara unik.
b. Atribut *simple*, yaitu atribut sederhana yang tidak dapat dibagi menjadi beberapa bagian.
c. Atribut komposit, yaitu atribut yang dapat dibagi lagi dalam beberapa bagian.
d. Atribut *single-valued*, yaitu atribut yang memiliki paling banyak satu nilai untuk setiap baris data.
e. Atribut *multivalued*: yaitu atribut yang dapat diisi dengan lebih satu nilai tetapi jenisnya sama.
f. Atribut turunan, yaitu atribut yang diperoleh dari pengolahan dari atribut lain yang berhubungan.
g. Atribut *Key*, yaitu atribut yang dapat dijadikan kunci untuk mencari data dalam relasi.

Dalam *database* ini, kami membuat enam entitas, yaitu entitas Nasabah, Rekening Bank, Akun *Mobile Banking,* Transaksi, Jenis Transaksi, dan Biaya Administrasi dengan ketentuan sebagai berikut.

Tabel Entitas dan Atribut dalam Rancangan *Database*

Entitas	Atribut	Key	Domain
Nasabah	ID_Nasabah	U, PK	Kumpulan empat digit karakter yang merepresentasikan ID Nasabah {N001, N002, …}.
	Nama_Nasabah		Kumpulan karakter string yang merepresentasikan nama nasabah.
	NoTelp_Nasabah		Kumpulan 12 digit angka yang merepresentasikan nomor telepon nasabah.

	Alamat_Nasabah		Kumpulan karakter string yang merepresentasikan alamat tempat tinggal nasabah.
	TTL_Nasabah		Kumpulan tanggal lahir nasabah dengan format DD-MM-YYYY.
Rekening_Bank	No_Rekening	U, PK	Kumpulan sepuluh digit angka yang merepresentasikan nomor rekening nasabah.
	Pin		Kumpulan enam digit angka yang merepresentasikan pin rekening nasabah.
	ID_Akun	U, FK	Kumpulan karakter yang merepresentasikan ID akun nasabah, misalnya {A001, A002, …}.
	ID_Nasabah	FK	Kumpulan empat digit karakter yang merepresentasikan ID Nasabah {N001, N002, …}.
	Saldo	FK	Kumpulan integer positif yang merepresentasikan saldo terkini milik nasabah.
Akun_MBanking	ID_Akun	U, PK	Kumpulan karakter yang merepresentasikan ID akun nasabah, misalnya {A001, A002, …}.
	ID_Nasabah	U, FK	Kumpulan lima digit karakter yang merepresentasikan ID nasabah {N001, N002, …}.

	Username	U	Kumpulan string alfanumerik dengan panjang antara 6 hingga 10 karakter yang merepresentasikan *username* nasabah untuk *log-in* aplikasi.
	Password		Kumpulan string alfanumerik dengan panjang enam karakter yang merepresentasikan *password* nasabah untuk *log-in* aplikasi.
Transaksi	ID_Transaksi	U, PK	Terdiri dari kumpulan 4 digit angka yang valid {0001, 0002, …}.
	ID_Jenis_Transaksi	U, FK	Kumpulan karakter yang merepresentasikan ID jenis transaksi, di mana {T0: Debit, T1: Kredit Transfer Antar Rekening, T2: Kredit Transfer Antar Bank, T3: Kredit Transfer Virtual Account, T4: Top-up}.
	Tanggal_Transaksi		Kumpulan tanggal nasabah melakukan transaksi dengan format DD-MM-YYYY.
	Nominal_Transaksi		Kumpulan integer positif yang merepresentasikan nominal transaksi yang dilakukan oleh nasabah.
	Saldo		Kumpulan integer positif yang merepresentasikan saldo terkini

	(Derived Attribute dari Saldo dan Nominal_Transaksi)		milik nasabah.
	ID_Biaya_Admin	U, FK	Kumpulan karakter yang merepresentasikan ID biaya administrasi, di mana {B0: Tanpa Biaya Admin, B1: BI Fast, B2: Realtime Online, B3: Top-Up}.
Jenis_Transaksi	ID_Jenis_Transaksi	U, PK	Kumpulan karakter yang merepresentasikan ID jenis transaksi, di mana {T0: Debit, T1: Kredit Transfer Antar Rekening, T2: Kredit Transfer Antar Bank, T3: Kredit Transfer Virtual Account, T4: Top-up}.
	Jenis_Transaksi		Kumpulan karakter string yang merepresentasikan jenis transaksi, misalnya {Debit, Kredit Transfer Rekening, Kredit Transfer Virtual Account, Kredit Top-up}.
Biaya_Admin	ID_Biaya_Admin	U, PK	Kumpulan karakter yang merepresentasikan ID biaya administrasi, di mana {B0: Tanpa Biaya Admin, B1: BI Fast, B2: Realtime Online, B3: Top-Up}.
	Jenis_Biaya_Admin		Kumpulan karakter string yang merepresentasikan nama biaya administrasi, misalnya {Tanpa Biaya Admin, BI Fast, Realtime

			Online, dan Top-Up}.
	Nominal_Biaya_Admin		Kumpulan integer positif yang merepresentasikan nominal biaya administrasi dari transaksi yang dilakukan nasabah.

Hubungan antar entitas, kardinalitas, deskripsi, dan jenis relasi dari rancangan *database* ini sebagai berikut.

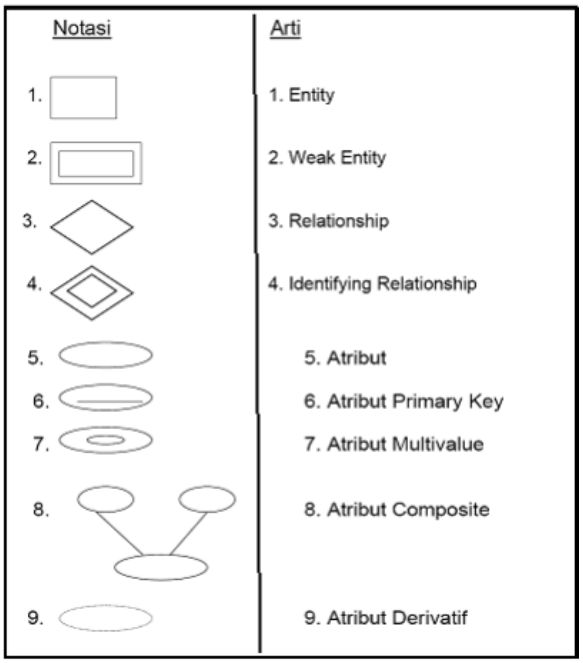
1. Satu nasabah harus memiliki satu atau lebih rekening bank.
2. Satu rekening bank hanya dapat dimiliki oleh satu nasabah.
3. Satu nasabah memiliki satu atau lebih akun mobile banking.
4. Setiap akun mobile banking hanya dapat dimiliki oleh satu nasabah.
5. Setiap nasabah yang memiliki akun mobile banking harus terhubung dengan rekening bank terkait yang dimiliki.
6. Satu rekening bank hanya dapat terhubung dengan satu akun mobile banking, dan sebaliknya.
7. Satu nasabah dapat melakukan satu atau lebih transaksi melalui akun mobile banking.
8. Setiap satu transaksi dilakukan oleh satu nasabah.
9. Setiap satu transaksi yang dilakukan nasabah akan dikenai satu jenis biaya administrasi yang bersesuaian.
10. Setiap transaksi yang dilakukan oleh nasabah, datanya akan tersimpan di rekening bank

2.2 ER *(Entity Relationship)* Diagram

Setelah dilakukan penyusunan rancangan *database*, akan ditunjukkan ER (*Entity Relationship*) diagram atau diagram hubungan entitas. Dengan diagram tersebut, hubungan atau relasi antar entitas beserta atribut-atribut dari rancangan *database* yang sudah disusun akan digambarkan dalam bentuk diagram secara detail. Dalam membuat ER diagram yang bersesuaian dengan hasil rancangan *database*, terdapat langkah-langkah yang dilakukan, sebagai berikut.

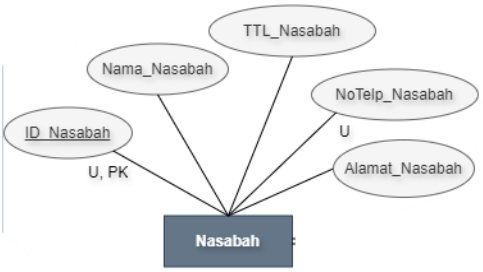
1. Mengidentifikasi entitas beserta atribut yang akan digunakan.

2. Mengidentifikasi dan menjelaskan relasi antar entitas. Pada tahap ini, akan diidentifikasi juga jenis kardinalitas atau derajat relasi yang bersesuaian.
3. Melengkapi diagram dengan beberapa karakteristik entitas, atribut, dan relasi lainnya seperti berikut.

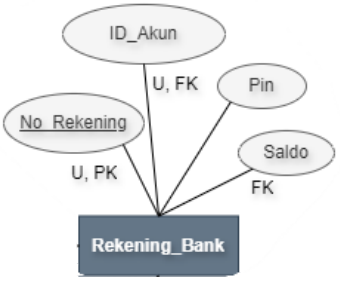


Berikut penjelasan mengenai tahapan penyusunan ER diagram berdasarkan hasil rancangan database untuk database perbankan.

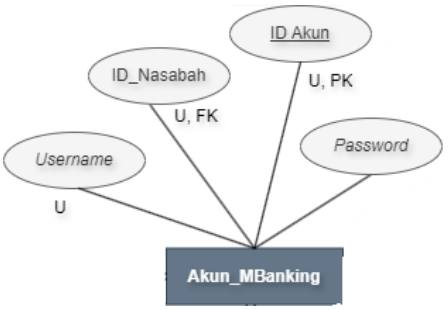
1. Mengidentifikasi entitas-entitas yang ada, lengkap dengan atribut dan *value set* untuk tiap atributnya.
    a. Entitas Nasabah yang terdiri dari atribut ID_Nasabah sebagai *primary key* dan bernilai unik, Nama_Nasabah, TTL_Nasabah, NoTelp_Nasabah, serta Alamat_Nasabah. Berikut potongan ER diagramnya.



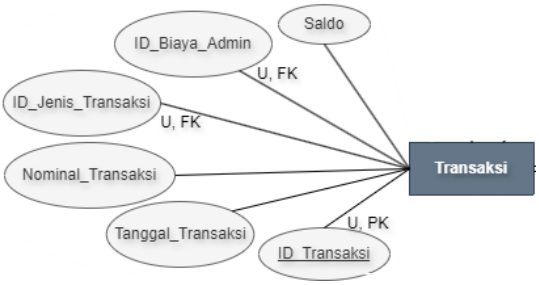
    b. Entitas Rekening_Bank yang terdiri dari atribut No_Rekening sebagai *primary key* dan bernilai unik, ID_Nasabah sebagai *foreign key* dan bernilai unik, Pin, dan Saldo sebagai *foreign key*. Berikut potongan ER diagramnya.



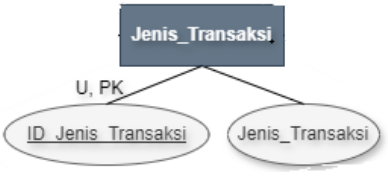
c. Entitas Akun_MBanking yang terdiri dari atribut ID_Akun sebagai *primary key* dan bernilai unik, ID_Nasabah sebagai *foreign key* dan bernilai unik, *Username* yang bernilai unik, dan *Password*. Berikut potongan ER diagramnya.



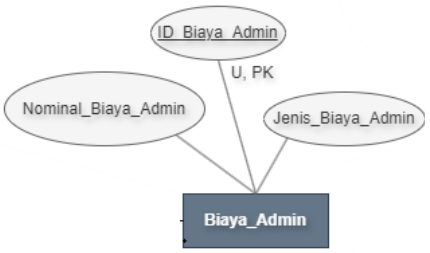
d. Entitas Transaksi yang terdiri dari atribut ID_Transaksi sebagai *primary key* dan bernilai unik, ID_Akun sebagai *foreign key* dan bernilai unik, ID_Biaya_Admin sebagai *foreign key* dan bernilai unik, ID_Biaya_Admin sebagai *foreign key* dan bernilai unik, Tanggal_Transaksi, Nominal_Transaksi, dan Saldo sebagai *foreign key*. Berikut potongan ER diagramnya.



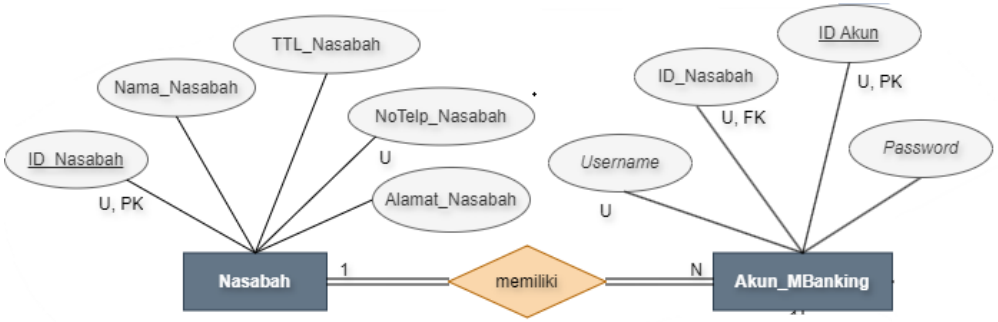
e. Entitas Jenis_Transaksi yang terdiri dari ID_Jenis_Transaksi sebagai *primary key* dan bernilai unik dan Jenis_Transaksi. Berikut potongan ER diagramnya.



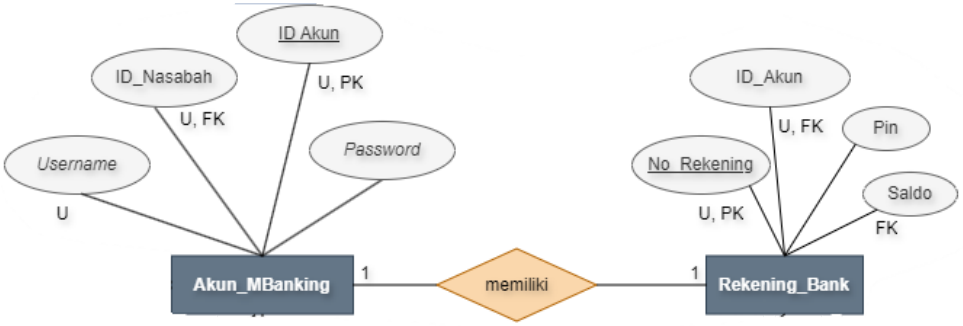
f. Entitas Biaya_Admin yang terdiri dari atribut ID_Biaya_Admin sebagai *primary key* dan bernilai unik, Jenis_Biaya_Admin, dan Nominal_Biaya_Admin. Berikut potongan ER diagramnya.



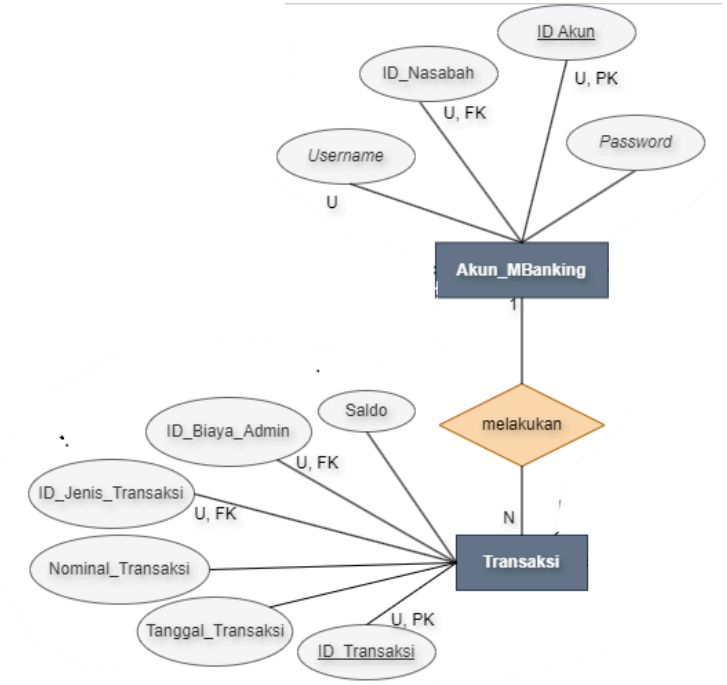
2. Mengidentifikasi relasi atau hubungan antar entitas yang telah disebutkan sebelumnya.
   a. Relasi Nasabah dan Akun_MBanking adalah satu **nasabah** harus memiliki satu atau lebih **akun *mobile banking*** (*One-to-Many*). Sebaliknya, satu **akun *mobile banking*** hanya dapat dimiliki oleh satu **nasabah**. Berikut potongan ER diagramnya.



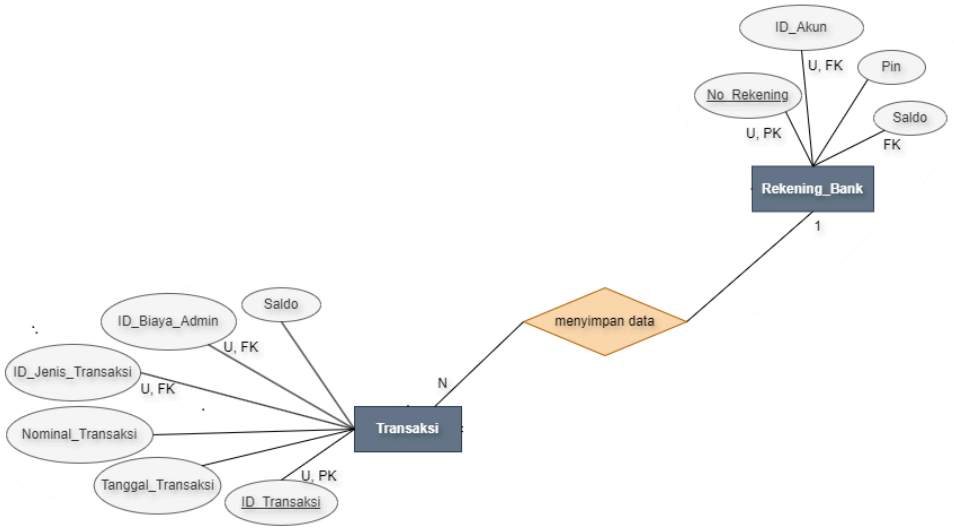
   b. Relasi Akun_MBanking dan Rekening_Bank adalah satu **rekening bank** hanya dapat memiliki satu **akun *mobile banking*** dengan nasabah yang bersesuaian (*One-to-One*). Berikut potongan ER diagramnya.



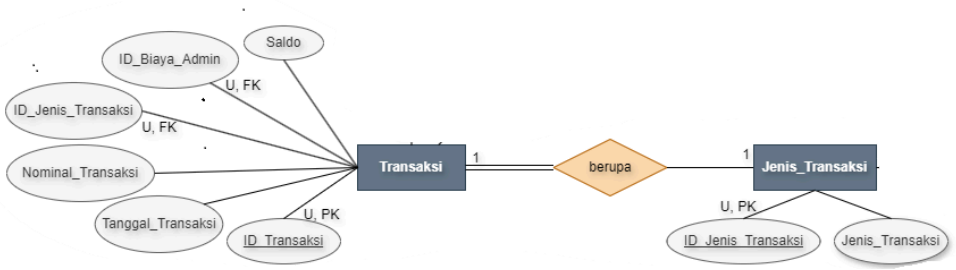
   c. Relasi Akun_MBanking dan Transaksi adalah satu **akun *mobile banking*** dapat melakukan lebih dari satu **transaksi** dengan nasabah yang bersesuaian (*One-to-Many*). Berikut potongan ER diagramnya.



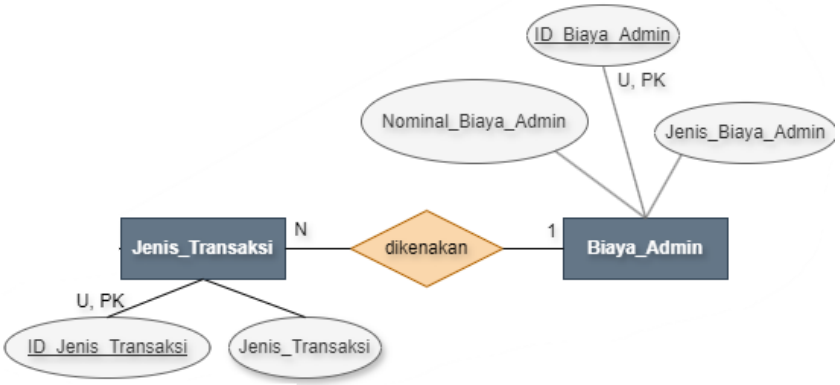
d. Relasi Rekening_Bank dan Transaksi adalah satu **rekening bank** dapat menyimpan lebih dari satu data **transaksi** dengan nasabah yang bersesuaian (*One-to-Many*). Berikut potongan ER diagramnya.



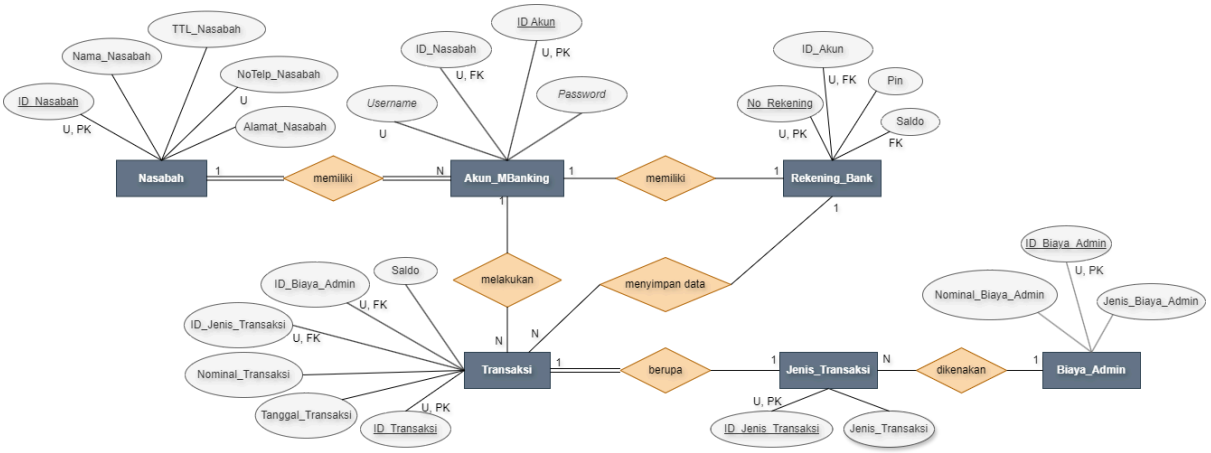
e. Relasi Transaksi dan Jenis_Transaksi adalah satu **transaksi** yang dilakukan oleh nasabah pasti berupa satu **jenis transaksi** (*One-to-One*). Berikut potongan ER diagramnya.



f. Relasi Jenis_Transaksi dan Biaya_Admin adalah satu atau lebih **jenis transaksi** akan dikenakan satu **biaya admin** yang bersesuaian (*Many-to-One*). Berikut potongan ER diagram.



3. Menggabungkan potongan-potongan tiap antar entitas sehingga diperoleh hasil akhir ER diagram lengkap sebagai berikut.



2.3 Tabel Relasional

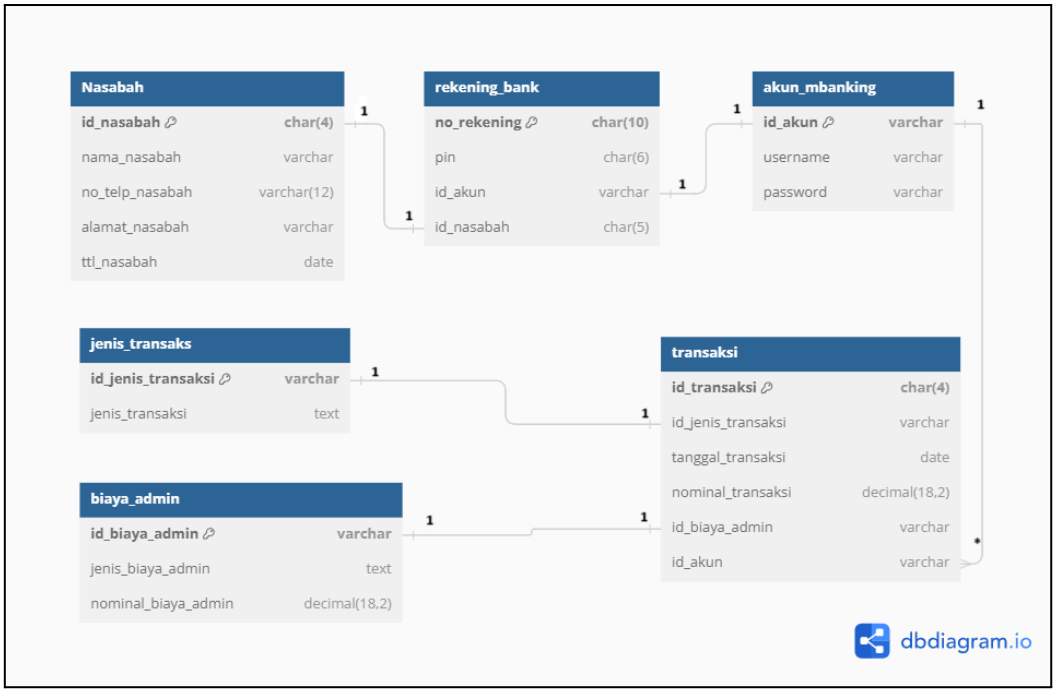
2.3.1 Tabel Relasional

Dalam membuat sebuah database, dibutuhkan sebuah struktur basis data untuk mempermudah membentuk basis data tersebut. Struktur basis data relasional sendiri mengatur data dalam tabel. Sebuah tabel tentunya terdiri dari baris dan kolom, di mana dalam

tabel relasional, barisnya disebut sebagai record dan kolomnya disebut field (atribut). Tentunya, hubungan antar tabel dalam struktur basis data relasional inilah yang akan menjadi pedoman ketika membuat basis data di dalam aplikasi yang kita inginkan, bersama dengan ER Diagram yang ada. Namun, dengan pertimbangan efektivitas, atribut saldo akan dihapus dikarenakan tidak adanya informasi saldo awal nasabah.

Tahap-tahap yang dibutuhkan dalam membuat tabel relasional adalah sebagai berikut:

1. Definisikan tujuan dari basis data.
2. Kumpulkan data dan atur tabel beserta dengan primary keys dan data classnya. Di dalam tabel, terdapat satu kolom yang berisi primary keys dan kolom lain yang berisi kelas data yang tentunya penting dalam pembuatan basis data nantinya.
3. Tentukan hubungan antar tabel.
4. Tentukan hubungan one-to-one, one-to-many, dan many-to-many dari semua tabel, seperti yang sudah didefinisikan di ER Diagram.



2.3.2 Set Data

Dalam pengolahan *database* ini, kamu menggunakan sebuah *dummy data* yang berisi 80 data transaksi dari beberapa nasabah di sebuah bank. Data ini memiliki 14 kolom berisi informasi nasabah dan informasi transaksi yang telah dilakukan. Data dapat diakses melalui tautan sebagai berikut:

https://docs.google.com/spreadsheets/d/1pq_6Y-3YZGmgpnW_eq0HIE0bTXxn7QMytVQGCw1PXHA/edit?usp=sharing

2.3.3 Normalisasi Data

Normalisasi adalah proses pengorganisasian data dalam sebuah *database* untuk mengurangi redundansi dan meningkatkan integritas data. Dengan menerapkan normalisasi, struktur tabel dalam *database* dioptimalkan sehingga data yang sama tidak disimpan di beberapa tempat. Proses ini mencakup beberapa langkah, biasanya dikenal sebagai bentuk normal *(normal forms)*, yang bertujuan untuk memastikan setiap tabel hanya berisi data yang berhubungan langsung dan tidak ada duplikasi data yang tidak perlu.

Pengurangan redundansi data sangat penting dalam sebuah rancangan *database* karena data yang redundan dapat menyebabkan inkonsistensi, kesulitan dalam pemeliharaan data, dan peningkatan kebutuhan penyimpanan. Redundansi data dapat terjadi ketika data yang sama disimpan dalam lebih dari satu tabel atau ketika sebuah tabel mengandung kolom yang bergantung pada kolom lain yang bukan merupakan *primary key*. Dalam *database* ini, kami akan melakukan normalisasi data hingga bentuk normal ketiga (3NF) sebagai berikut.

1. Bentuk Normal Pertama (1NF)

   Pada data ini, data sudah dalam bentuk normal pertama karena setiap atribut bernilai tunggal untuk setiap baris dan tidak ada set atribut yang berulang-ulang atau atribut yang bernilai ganda.

2. Bentuk Normal Kedua (2NF)

   Untuk membuat bentuk normal kedua, maka atribut bukan *key* harus bergantung secara fungsi pada *primary key.* Maka, akan dibuat enam tabel untuk membuat bentuk normal kedua untuk data ini sebagai berikut.

Tabel 1. Informasi Nasabah

ID_Nasabah	Nama_Nasabah	NoTelp_Nasabah	Alamat_Nasabah	TTL_Nasabah
N001	Agus Santoso	081389765432	Jalan Srengseng Sawah, Jakarta Selatan	27-10-1998
N002	Budi Hartono	081234567890	Jalan Sudirman, Jakarta Pusat	15-06-1985
N003	Citra Dewi	081298765432	Jalan Kebon Jeruk, Jakarta Barat	23-12-1990
N004	Ivan Syahputra	082345678901	Jalan Kemang, Jakarta Selatan	05-05-1989
N005	Muhammad Ilham	081789012345	Jalan Kalimalang, Jakarta Timur	18-12-1986
N006	Kartika Dewi	081678901234	Jalan Senayan, Jakarta Selatan	14-02-1993
N007	Nia Ramadhani	081890123456	Jalan Cikini, Jakarta Pusat	07-07-1997
N008	Oka Setiawan	082567890123	Jalan Pramuka, Jakarta Timur	25-03-1990

Tabel 2. Informasi Rekening Nasabah

No_Rekening	Pin	ID_Akun	ID_Nasabah
4327819903	889212	A001	N001
4321765412	998877	A002	N002
4321928752	776655	A003	N003
4327618908	889220	A004	N004
4327819913	889224	A005	N005
4327819911	889222	A006	N006
4309875617	889225	A007	N007
4327618915	889226	A008	N008

Tabel 3. Informasi Akun *Mobile Banking* Nasabah

ID_Akun	Username	Password
A001	agus123	27agus
A002	budi21a	15budi
A003	citra789	23citra
A004	ivnsy89	ivaa05
A005	mhmil86	ilham8
A006	krtkdw93	93tika
A007	niaram97	niara7
A008	okast90	okase2

Tabel 4. Transaksi*

ID_Akun	ID_Transaksi	ID_Jenis_Transaksi	Tanggal_Transaksi	Nominal_Transaksi	ID_Biaya_Admin
A001	0001	T3	02-01-2022	57036	B3
A002	0002	T3	12-01-2022	26206	B3
A003	0003	T3	15-01-2022	73956	B3
A002	0004	T2	22-01-2022	300000	B2
A002	0005	T1	23-01-2022	100000	B0
A003	0006	T0	27-01-2022	1000000	B0
A001	0007	T0	03-02-2022	50000	B0
A001	0008	T4	05-02-2022	30000	B3
A003	0009	T1	16-02-2022	50000	B0
A001	0010	T2	19-02-2022	250000	B2
A001	0011	T2	28-02-2022	150000	B2
A001	0012	T4	04-03-2022	20000	B3
A003	0013	T2	05-03-2022	150000	B1
A001	0014	T0	05-03-2022	2000000	B0
A003	0015	T1	09-03-2022	130000	B0
A002	0016	T2	11-03-2022	200000	B1
A001	0017	T2	22-03-2022	500000	B1
A001	0018	T0	25-03-2022	1000000	B0
A002	0019	T2	31-03-2022	100000	B2
A002	0020	T4	02-04-2022	80000	B3
A001	0021	T3	09-04-2022	50000	B3

*Tabel transaksi yang ditampilkan hanya sebagian untuk alasan efisiensi.

Tabel 5. Jenis Transaksi

ID_Jenis_Transaksi	Jenis_Transaksi
T0	Debit
T1	Kredit Transfer Antar Rekening
T2	Kredit Transfer Antar Bank
T3	Kredit Transfer Virtual Account
T4	Top-Up

Tabel 6. Biaya Administrasi

ID_Biaya_Admin	Jenis_Biaya_Admin	Nominal_Biaya_Admin
B0	Tanpa Biaya Admin	0
B1	BI Fast	2500
B2	Realtime Online	6500
B3	Top-Up	1000

3. Bentuk Normal Ketiga (3NF)

Untuk membuat bentuk normal ketiga dalam sebuah data, maka data tidak boleh memiliki *transitive functional dependencies.* Untuk Tabel 1 hingga Tabel 6 yang telah dibentuk sebelumnya tidak memiliki *transitive functional dependencies,* maka telah memenuhi bentuk normal ketiga. Maka, terdapat enam tabel untuk bentuk normal ketiga dalam data ini.

BAB III
IMPLEMENTASI DAN PEMBAHASAN

Pada bagian ini, akan dijelaskan secara rinci mengenai implementasi SQL dan GUI yang telah dilakukan pada basis data kami. Sebelum masuk pada bagian implementasi SQL dan GUI, akan dijelaskan mengenai tahapan awal, yaitu persiapan dan pembuatan basis data dengan bahasa Python di Google Collaboratory.

Tahap pertama yang perlu dilakukan adalah *import packages* dan *library* yang diperlukan selama proses *running* implementasi SQL dan GUI. Berikut daftar *packages* dan *library* yang digunakan.

```
[ ]  import pandas as pd
     import sqlite3
     from io import StringIO
     import csv
     from ipywidgets import Layout, widgets, Style
     from datetime import datetime
     from IPython.display import display, clear_output
```

- *pandas* digunakan untuk menjadikan struktur data sebagai DataFrame kemudian mengolahnya.
- *sqlite3* digunakan untuk menjalankan perintah SQL dan berinteraksi dengan basis data yang ada.
- *io* dan *StringIO* digunakan untuk mendukung proses *input/output* data ataupun file.
- *csv* digunakan untuk membaca file data bertipe csv.
- *ipywidgets* digunakan untuk menyediakan *user interface* yang interaktif.
- *datetime* digunakan untuk menyediakan dan mengelola data berupa tanggal.
- *IPython.display* digunakan untuk menampilkan berbagai jenis data.

Kemudian masuk pada bagian membuat basis data dan membuat tabel-tabel yang mendukung sistem basis data perbankan lengkap dengan atribut dan sifat atribut yang bersesuaian akan melibatkan definisi skema basis data, relasi antar tabel, serta pengaturan *primary key* dan *foreign key* untuk memastikan integritas data, seperti berikut.

- Membuat Sistem Basis Data dan Membuka Koneksi Basis Data

```
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()
```

  Menyambungkan ke basis data `perbankan.db` yang telah dibuat dan kursor untuk menjalankan perintah SQL yang akan dilakukan.
- Mengaktifkan dukungan *foreign key*

```
# Mengaktifkan dukungan foreign key
c.execute('PRAGMA foreign_keys = ON')
```

Dukungan *foreign key* diaktifkan. Hal ini dikarenakan ada atribut pada beberapa entitas, terdapat atribut yang berperan sebagai *foreign key* supaya relasi antar entitas dapat berlaku.

- Membuat Tabel Nasabah

```
c.execute('''CREATE TABLE Nasabah (
             ID_Nasabah CHAR(4) PRIMARY KEY NOT NULL,
             Nama_Nasabah TEXT,
             NoTelp_Nasabah CHAR,
             Alamat_Nasabah CHAR,
             TTL_Nasabah CHAR(10))''')
```

Terdapat 5 atribut pada tabel Nasabah, yaitu:

1. `ID_Nasabah` : *primary key* dengan tipe data `CHAR(4)`.
2. `Nama_Nasabah` : teks untuk menyimpan nama nasabah.
3. `NoTelp_Nasabah` : nomor telepon nasabah.
4. `Alamat_Nasabah` : alamat nasabah.
5. `TTL_Nasabah` : tanggal lahir nasabah dengan tipe data `CHAR(10`.

- Membuat Tabel Rekening_Bank

```
c.execute('''CREATE TABLE Rekening_Bank (
             No_Rekening INTEGER PRIMARY KEY NOT NULL,
             Pin INTEGER NOT NULL,
             ID_Nasabah CHAR(4) NOT NULL,
             ID_Akun CHAR(4) NOT NULL)''')
```

Terdapat 4 atribut pada tabel Rekening_Bank, yaitu:

1. `No_Rekening` : *primary key* dengan tipe data integer dan tidak bernilai null.
2. `Pin` : integer untuk menyimpan innama nasabah.
3. `ID_Nasabah` : *foreign key* dengan tipe data `CHAR(4)` mengenai ID nasabah..
4. `ID_Akun` : *foreign key* dengan tipe data `CHAR(4)` mengenai ID akun *mobile banking* nasabah.

- Membuat Tabel Akun_MBanking

```
c.execute('''CREATE TABLE Akun_MBanking (
             ID_Akun CHAR(4) PRIMARY KEY NOT NULL,
             Username CHAR(10),
             Password CHAR(6))''')
```

Terdapat 3 atribut pada tabel Akun_MBanking, yaitu:

1. `ID_Akun` : *primary key* dengan tipe data `CHAR(4)` mengenai ID akun *mobile banking* nasabah dan tidak bernilai null.

2. `Username` : CHAR(10) untuk menyimpan *username* akun *mobile banking* nasabah.
3. `Password` : CHAR(6) untuk menyimpan *password* akun *mobile banking* nasabah.

- Membuat Tabel Transaksi

```
c.execute('''CREATE TABLE Transaksi (
             ID_Transaksi CHAR(4) PRIMARY KEY NOT NULL,
             ID_Akun CHAR(4) NOT NULL,
             ID_Jenis_Transaksi CHAR(2) NOT NULL,
             Tanggal_Transaksi CHAR(10) NOT NULL,
             Nominal_Transaksi INTEGER,
             ID_Biaya_Admin CHAR(2))''')
```

Terdapat 6 atribut pada tabel Transaksi, yaitu:

1. `ID_Transaksi` : *primary key* dengan tipe data `CHAR(4)` mengenai ID transaksi nasabah dan tidak bernilai null.
2. `ID_Akun` : *foreign key* dengan tipe data `CHAR(4)` mengenai ID akun *mobile banking* nasabah dan tidak bernilai null.
3. `ID_Jenis_Transaksi` : *foreign key* dengan tipe data `CHAR(2)` mengenai ID jenis transaksi nasabah dan tidak bernilai null.
4. `Tanggal_Transaksi` : `CHAR(10)` dengan format dd-mm-yyyy untuk menyimpan tanggal transaksi yang dilakukan nasabah dan tidak bernilai null.
5. `Nominal_Transaksi` : tipe data integer untuk menyimpan nominal transaksi yang dilakukan nasabah.
6. `ID_Biaya_Admin` : *foreign key* dengan tipe data `CHAR(2)` mengenai ID biaya administrasi yang dikenakan pada nasabah saat melakukan transaksi.

- Membuat Tabel BIaya_Admin

```
c.execute('''CREATE TABLE Biaya_Admin (
             ID_Biaya_Admin CHAR(2) PRIMARY KEY NOT NULL,
             Jenis_Biaya_Admin CHAR NOT NULL,
             Nominal_Biaya_Admin INTEGER)''')
```

Terdapat 3 atribut pada tabel Biaya_Admin, yaitu:

1. `ID_Biaya_Admin` : *primary key* dengan tipe data `CHAR(2)` mengenai ID biaya administrasi yang dikenakan pada nasabah saat melakukan transaksi.
2. `Jenis_Biaya_Admin` : tipe data `CHAR` mengenai jenis biaya admin yang dikenakan pada nasabah saat melakukan transaksi.
3. `Nominal_Biaya_Admin` : tipe data integer untuk menyimpan nominal biaya admin yang dikenakan pada nasabah saat melakukan transaksi.

- Membuat Tabel Jenis_Transaksi

```
c.execute('''CREATE TABLE Jenis_Transaksi (
             ID_Jenis_Transaksi CHAR(2) PRIMARY KEY NOT NULL,
             Jenis_Transaksi CHAR NOT NULL)''')
```

Terdapat 2 atribut pada tabel Jenis_Transaksi, yaitu:

1. `ID_Jenis_Transaksi` : *primary key* dengan tipe data `CHAR(2)` mengenai ID jenis transaksi nasabah dan tidak bernilai null.
2. `Jenis_Transaksi` : tipe data `CHAR` mengenai jenis transaksi yang dilakukan oleh nasabah.

- Menyimpan dan Menutup Koneksi Basis Data

```
conn.commit()
conn.close()
```

Setelah membuat basis data `perbankan.db`, *dummy data* yang sudah disusun diunggah melalui github.com sehingga dapat terhubung langsung dengan *notebook* implementasi SQL dan GUI, seperti berikut.

```
Akun_Mbanking = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Akun_MBanking.csv")
Biaya_Admin = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Biaya_Admin.csv")
Nasabah = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Nasabah.csv")
Jenis_Transaksi = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Jenis_Transaksi%20(1).csv")
Transaksi = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Transaksi(fix).csv")
Rekening_Bank = pd.read_csv("https://raw.githubusercontent.com/Fyodorrrr/Database/main/Data-Perbankan%20-%20Rekening_Bank.csv")
```

Selanjutnya, dilakukan *insert dummy data* untuk tiap tabel yang telah dibuat sebelumnya menggunakan fungsi *insert* seperti berikut.

- Tabel Akun_MBanking

```
[ ]  # Hubungkan ke database SQLite
     MyDB = sqlite3.connect('perbankan.db')
     cursor = MyDB.cursor()

     # Baca file CSV
     for row in Akun_Mbanking.itertuples(index=False):
       print(row)
       if len(row) == 3:
         cursor.execute("INSERT INTO Akun_MBanking (ID_Akun, Username, Password) VALUES (?, ?, ?)", tuple(row))
       else:
         print(f"Baris tidak valid (kolom tidak sesuai): {row}")

     # Komit perubahan dan tutup koneksi
     MyDB.commit()
     MyDB.close()
```

```
Pandas(ID_Akun='A001', Username='agus123', Password='27agus')
Pandas(ID_Akun='A002', Username='budi21a', Password='15budi')
Pandas(ID_Akun='A003', Username='citra789', Password='23citra')
Pandas(ID_Akun='A004', Username='ivnsy89', Password='ivaa05')
Pandas(ID_Akun='A005', Username='mhmil86', Password='ilham8')
Pandas(ID_Akun='A006', Username='krtkdw93', Password='93tika')
Pandas(ID_Akun='A007', Username='niaram97', Password='niara7')
Pandas(ID_Akun='A008', Username='okast90', Password='okase2')
```

- Tabel Biaya_Admin

```
[ ]  # Hubungkan ke database SQLite
     MyDB = sqlite3.connect('perbankan.db')
     cursor = MyDB.cursor()

     # Baca file CSV
     for row in Biaya_Admin.itertuples(index=False):
       print(row)
       if len(row) == 3:
         cursor.execute("INSERT INTO Biaya_Admin (ID_Biaya_Admin, Jenis_Biaya_Admin, Nominal_Biaya_Admin) VALUES (?, ?, ?)", tuple(row))
       else:
           print(f"Baris tidak valid (kolom tidak sesuai): {row}")
     # Komit perubahan dan tutup koneksi
     MyDB.commit()
     MyDB.close()
```

```
Pandas(ID_Biaya_Admin='B0', Jenis_Biaya_Admin='Tanpa Biaya Admin', Nominal_Biaya_Admin=0)
Pandas(ID_Biaya_Admin='B1', Jenis_Biaya_Admin='BI Fast', Nominal_Biaya_Admin=2500)
Pandas(ID_Biaya_Admin='B2', Jenis_Biaya_Admin='Realtime Online', Nominal_Biaya_Admin=6500)
Pandas(ID_Biaya_Admin='B3', Jenis_Biaya_Admin='Top-Up', Nominal_Biaya_Admin=1000)
```

- Tabel Nasabah

```
[ ]  # Hubungkan ke database SQLite
     MyDB = sqlite3.connect('perbankan.db')
     cursor = MyDB.cursor()

     # Baca file CSV
     for row in Nasabah.itertuples(index=False):
         print(row)
         if len(row) == 5:
             cursor.execute("INSERT INTO Nasabah (ID_Nasabah, Nama_Nasabah, NoTelp_Nasabah, Alamat_Nasabah, TTL_Nasabah) VALUES (?, ?, ?, ?, ?)", tuple(row))
         else:
             print(f"Baris tidak valid (kolom tidak sesuai): {row}")

     # Komit perubahan dan tutup koneksi
     MyDB.commit()
     MyDB.close()
```

```
Pandas(ID_Nasabah='N001', Nama_Nasabah='Agus Santoso', NoTelp_Nasabah=81389765432, Alamat_Nasabah='Jalan Srengseng Sawah, Jakarta Selatan', TTL_Nasabah='27-10-19
Pandas(ID_Nasabah='N002', Nama_Nasabah='Budi Hartono', NoTelp_Nasabah=81234567890, Alamat_Nasabah='Jalan Sudirman, Jakarta Pusat', TTL_Nasabah='15-06-1985')
Pandas(ID_Nasabah='N003', Nama_Nasabah='Citra Dewi', NoTelp_Nasabah=81298765432, Alamat_Nasabah='Jalan Kebon Jeruk, Jakarta Barat', TTL_Nasabah='23-12-1990')
Pandas(ID_Nasabah='N004', Nama_Nasabah='Ivan Syahputra', NoTelp_Nasabah=82345678901, Alamat_Nasabah='Jalan Kemang, Jakarta Selatan', TTL_Nasabah='05-05-1989')
Pandas(ID_Nasabah='N005', Nama_Nasabah='Muhammad Ilham', NoTelp_Nasabah=81789012345, Alamat_Nasabah='Jalan Kalimalang, Jakarta Timur', TTL_Nasabah='18-12-1986')
Pandas(ID_Nasabah='N006', Nama_Nasabah='Kartika Dewi', NoTelp_Nasabah=81678901234, Alamat_Nasabah='Jalan Senayan, Jakarta Selatan', TTL_Nasabah='14-02-1993')
Pandas(ID_Nasabah='N007', Nama_Nasabah='Nia Ramadhani', NoTelp_Nasabah=81890123456, Alamat_Nasabah='Jalan Cikini, Jakarta Pusat', TTL_Nasabah='07-07-1997')
Pandas(ID_Nasabah='N008', Nama_Nasabah='Oka Setiawan', NoTelp_Nasabah=82567890123, Alamat_Nasabah='Jalan Pramuka, Jakarta Timur', TTL_Nasabah='25-03-1990')
```

- Tabel Rekening_Bank

```
[ ]  # Hubungkan ke database SQLite
     MyDB = sqlite3.connect('perbankan.db')
     cursor = MyDB.cursor()

     # Baca file CSV
     for row in Rekening_Bank.itertuples(index=False):
         print(row)
         if len(row) == 4:
             cursor.execute("INSERT INTO Rekening_Bank (ID_Nasabah, No_Rekening, Pin, ID_Akun) VALUES (?, ?, ?, ?)", tuple(row))
         else:
             print(f"Baris tidak valid (kolom tidak sesuai): {row}")

     # Komit perubahan dan tutup koneksi
     MyDB.commit()
     MyDB.close()
```

```
Pandas(ID_Nasabah='N001', No_Rekening=4327819903, Pin=889212, ID_Akun='A001')
Pandas(ID_Nasabah='N002', No_Rekening=4321765412, Pin=998877, ID_Akun='A002')
Pandas(ID_Nasabah='N003', No_Rekening=4321928752, Pin=776655, ID_Akun='A003')
Pandas(ID_Nasabah='N004', No_Rekening=4327618908, Pin=889220, ID_Akun='A004')
Pandas(ID_Nasabah='N005', No_Rekening=4327819913, Pin=889224, ID_Akun='A005')
Pandas(ID_Nasabah='N006', No_Rekening=4327819911, Pin=889222, ID_Akun='A006')
Pandas(ID_Nasabah='N007', No_Rekening=4309875617, Pin=889225, ID_Akun='A007')
Pandas(ID_Nasabah='N008', No_Rekening=4327618915, Pin=889226, ID_Akun='A008')
```

- Tabel Transaksi

```
[ ] # Hubungkan ke database SQLite
    MyDB = sqlite3.connect('perbankan.db')
    cursor = MyDB.cursor()

    # Baca file CSV
    for row in Transaksi.itertuples(index=False):
        print(row)
        if len(row) == 6:
            cursor.execute("INSERT INTO Transaksi (ID_Akun, ID_Transaksi, ID_Jenis_Transaksi, Tanggal_Transaksi, Nominal_Transaksi, ID_Biaya_Admin) VALUES (?, ?, ?,
        else:
            print(f"Baris tidak valid (kolom tidak sesuai): {row}")

    # Komit perubahan dan tutup koneksi
    MyDB.commit()
    MyDB.close()
```

```
Pandas(ID_Akun='A001', ID_Transaksi=1, ID_Jenis_Transaksi='T3', Tanggal_Transaksi='02-01-2022', Nominal_Transaksi=57036, ID_Biaya_Admin='B3')
Pandas(ID_Akun='A002', ID_Transaksi=2, ID_Jenis_Transaksi='T3', Tanggal_Transaksi='12-01-2022', Nominal_Transaksi=26206, ID_Biaya_Admin='B3')
Pandas(ID_Akun='A003', ID_Transaksi=3, ID_Jenis_Transaksi='T3', Tanggal_Transaksi='15-01-2022', Nominal_Transaksi=73956, ID_Biaya_Admin='B3')
Pandas(ID_Akun='A002', ID_Transaksi=4, ID_Jenis_Transaksi='T2', Tanggal_Transaksi='22-01-2022', Nominal_Transaksi=300000, ID_Biaya_Admin='B2')
Pandas(ID_Akun='A002', ID_Transaksi=5, ID_Jenis_Transaksi='T1', Tanggal_Transaksi='23-01-2022', Nominal_Transaksi=100000, ID_Biaya_Admin='B0')
Pandas(ID_Akun='A003', ID_Transaksi=6, ID_Jenis_Transaksi='T0', Tanggal_Transaksi='27-01-2022', Nominal_Transaksi=1000000, ID_Biaya_Admin='B0')
Pandas(ID_Akun='A001', ID_Transaksi=7, ID_Jenis_Transaksi='T0', Tanggal_Transaksi='03-02-2022', Nominal_Transaksi=50000, ID_Biaya_Admin='B0')
Pandas(ID_Akun='A001', ID_Transaksi=8, ID_Jenis_Transaksi='T4', Tanggal_Transaksi='05-02-2022', Nominal_Transaksi=30000, ID_Biaya_Admin='B3')
Pandas(ID_Akun='A003', ID_Transaksi=9, ID_Jenis_Transaksi='T1', Tanggal_Transaksi='16-02-2022', Nominal_Transaksi=50000, ID_Biaya_Admin='B0')
Pandas(ID_Akun='A001', ID_Transaksi=10, ID_Jenis_Transaksi='T2', Tanggal_Transaksi='19-02-2022', Nominal_Transaksi=250000, ID_Biaya_Admin='B2')
Pandas(ID_Akun='A001', ID_Transaksi=11, ID_Jenis_Transaksi='T2', Tanggal_Transaksi='28-02-2022', Nominal_Transaksi=150000, ID_Biaya_Admin='B2')
```

- Tabel Jenis_Transaksi

```
[ ] # Hubungkan ke database SQLite
    MyDB = sqlite3.connect('perbankan.db')
    cursor = MyDB.cursor()

    # Baca file CSV
    for row in Jenis_Transaksi.itertuples(index=False):
        print(row)
        if len(row) == 2:
            cursor.execute("INSERT INTO Jenis_Transaksi (ID_Jenis_Transaksi, Jenis_Transaksi) VALUES (?, ?)", (row[0], row[1]))
        else:
            print(f"Baris tidak valid (kolom tidak sesuai): {row}")

    # Komit perubahan dan tutup koneksi
    MyDB.commit()
    MyDB.close()
```

```
Pandas(ID_Jenis_Transaksi='T0', Jenis_Transaksi='Debit')
Pandas(ID_Jenis_Transaksi='T1', Jenis_Transaksi='Kredit Transfer Antar Rekening')
Pandas(ID_Jenis_Transaksi='T2', Jenis_Transaksi='Kredit Transfer Antar Bank')
Pandas(ID_Jenis_Transaksi='T3', Jenis_Transaksi='Kredit Transfer Virtual Account')
Pandas(ID_Jenis_Transaksi='T4', Jenis_Transaksi='Top-Up')
```

3.1 Implementasi SQL dan *Testing*

Pada subbab implementasi SQL ini, akan dijelaskan proses implementasi basis data menggunakan *Structured Query Language* (SQL), khususnya implementasi mengenai operasi-operasi manipulasi data, seperti *create, insert, update, select,* dan *join* yang digunakan untuk mengelola data di dalam basis data.

Berikut tahapan implementasi SQL serta pembahasan untuk fungsi-fungsi utama pada SQL.

1. Fungsi Insert

Pada implementasi fungsi *insert* ini, kami mencoba untuk menambahkan satu data nasabah baru pada tabel Nasabah. Berikut rinciannya.

- Menghubungkan ke basis data yang sudah ada.

```
# Hubungkan ke database SQLite
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()
```

- Menyiapkan data yang akan dimasukkan bersesuaian dengan atribut-atribut yang ada di tabel Nasabah.

```
# Data yang akan dimasukkan
data = {
    'ID_Nasabah': 'N009',
    'Nama_Nasabah': 'Lorem Ipsum',
    'NoTelp_Nasabah': '08123456789',
    'Alamat_Nasabah': 'Jalan Mawar, Jakarta Utara',
    'TTL_Nasabah': '31-12-1995'
}
```

- Membuat *query* fungsi *insert.*

```
# Membuat query INSERT
columns = ', '.join(data.keys())
placeholders = ', '.join('?' for _ in data)
sql = f"INSERT INTO Nasabah ({columns}) VALUES ({placeholders})"
values = tuple(data.values())
```

  - `columns` untuk menyimpan nama-nama kolom yang akan diisi.
  - `placeholders` adalah tanda tanya (`?`) yang digunakan sebagai *placeholder* untuk nilai-nilai yang akan dimasukkan.
  - `sql` berisi *query* `INSERT` yang dirangkai dengan menggunakan nama kolom dan *placeholder.*
  - `values` adalah tuple dari nilai-nilai yang akan dimasukkan.

- Menjalankan *query* fungsi *insert*.

```
# Menjalankan query INSERT
c.execute(sql, values)
conn.commit()
print("Data berhasil dimasukkan.")
```

Query `INSERT` dieksekusi dengan menggunakan cursor (`c.execute`). Setelah itu, perubahan disimpan ke basis data `perbankan.db` dan akan dicetak "Data berhasil dimasukkan." untuk mengonfirmasi keberhasilan operasi.

- Menutup koneksi basis data.

```
# Menutup koneksi
conn.close()
```

- Menampilkan data dalam tabel Nasabah.

```
[ ]
# Hubungkan ke database SQLite
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()

# Menjalankan query untuk mengambil semua data dari tabel Nasabah
c.execute('SELECT * FROM Nasabah')

# Mengambil semua baris hasil query
rows = c.fetchall()

# Menampilkan hasil query
print("Data dalam tabel Nasabah:")
for row in rows:
    print(row)

# Menutup koneksi
conn.close()
```

```
Data dalam tabel Nasabah:
('N001', 'Agus Santoso', '81389765432', 'Jalan Srengseng Sawah, Jakarta Selatan', '27-10-1998')
('N002', 'Budi Hartono', '81234567890', 'Jalan Sudirman, Jakarta Pusat', '15-06-1985')
('N003', 'Citra Dewi', '81298765432', 'Jalan Kebon Jeruk, Jakarta Barat', '23-12-1990')
('N004', 'Ivan Syahputra', '82345678901', 'Jalan Kemang, Jakarta Selatan', '05-05-1989')
('N005', 'Muhammad Ilham', '81789012345', 'Jalan Kalimalang, Jakarta Timur', '18-12-1986')
('N006', 'Kartika Dewi', '81678901234', 'Jalan Senayan, Jakarta Selatan', '14-02-1993')
('N007', 'Nia Ramadhani', '81890123456', 'Jalan Cikini, Jakarta Pusat', '07-07-1997')
('N008', 'Oka Setiawan', '82567890123', 'Jalan Pramuka, Jakarta Timur', '25-03-1990')
('N009', 'Lorem Ipsum', '08123456789', 'Jalan Mawar, Jakarta Utara', '31-12-1995')
```

Dapat dilihat bahwa data yang ingin dimasukkan ke tabel Nasabah, telah disimpan.

2. Fungsi Update

Pada implementasi fungsi *update* ini, kami mencoba untuk memperbarui dua nilai pada tabel Nasabah. Berikut rinciannya.

- Menghubungkan ke basis data yang sudah ada.

```
# Hubungkan ke database SQLite
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()
```

- Menyiapkan data yang akan diperbarui bersesuaian dengan atribut-atribut yang ada di tabel Nasabah.

```
# Data yang akan diperbarui
data = {
    'Nama_Nasabah': 'Joshua Hong',
    'NoTelp_Nasabah': '08198765432'
}
```

- Menentukan kondisi untuk memperbarui data.

```
condition = "ID_Nasabah = 'N009'"
```

- Membuat *query* fungsi *update.*

```
# Membuat query UPDATE
set_clause = ', '.join(f"{col} = ?" for col in data.keys())
sql = f"UPDATE Nasabah SET {set_clause} WHERE {condition}"
values = tuple(data.values())
```

- `set_clause` untuk menyimpan pasangan kolom dan *placeholder* yang akan diperbarui.
- `sql` berisi *query* `UPDATE` yang dirangkai dengan menggunakan nama kolom yang akan diperbarui.
- `values` adalah tuple dari nilai-nilai baru yang akan diperbarui.

- Menjalankan *query* fungsi *insert*.

```
# Menjalankan query UPDATE
c.execute(sql, values)
conn.commit()
print("Data berhasil diperbarui.")
```

Query `UPDATE` dieksekusi dengan menggunakan cursor (`c.execute`). Setelah itu, perubahan disimpan ke basis data `perbankan.db` dan akan dicetak "Data berhasil dimasukkan." untuk mengonfirmasi keberhasilan operasi.

- Menutup koneksi basis data.

```
# Menutup koneksi
conn.close()
```

- Menampilkan data dalam tabel Nasabah.

```
[ ]
# Hubungkan ke database SQLite
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()

# Menjalankan query untuk mengambil semua data dari tabel Nasabah
c.execute('SELECT * FROM Nasabah')

# Mengambil semua baris hasil query
rows = c.fetchall()

# Menampilkan hasil query
print("Data dalam tabel Nasabah:")
for row in rows:
    print(row)

# Menutup koneksi
conn.close()
```

```
Data dalam tabel Nasabah:
('N001', 'Agus Santoso', '81389765432', 'Jalan Srengseng Sawah, Jakarta Selatan', '27-10-1998')
('N002', 'Budi Hartono', '81234567890', 'Jalan Sudirman, Jakarta Pusat', '15-06-1985')
('N003', 'Citra Dewi', '81298765432', 'Jalan Kebon Jeruk, Jakarta Barat', '23-12-1990')
('N004', 'Ivan Syahputra', '82345678901', 'Jalan Kemang, Jakarta Selatan', '05-05-1989')
('N005', 'Muhammad Ilham', '81789012345', 'Jalan Kalimalang, Jakarta Timur', '18-12-1986')
('N006', 'Kartika Dewi', '81678901234', 'Jalan Senayan, Jakarta Selatan', '14-02-1993')
('N007', 'Nia Ramadhani', '81890123456', 'Jalan Cikini, Jakarta Pusat', '07-07-1997')
('N008', 'Oka Setiawan', '82567890123', 'Jalan Pramuka, Jakarta Timur', '25-03-1990')
('N009', 'Joshua Hong', '08198765432', 'Jalan Mawar, Jakarta Utara', '31-12-1995')
```

Dapat dilihat bahwa data Nasabah dengan ID_Nasabah N009, sudah perbarui Nama_Nasabah dan TTL_Nasabah.

3. Fungsi Select

Pada implementasi fungsi *select,* kelompok kami mencoba untuk menyeleksi sekaligus menampilkan transaksi debit atau `T0` dengan rincian sebagai berikut.

- Menggunakan fungsi *select* dan memanggil tabel nasabah untuk memilih nasabah yang bernama `Agus Santoso`. Setelah itu, *output* akan disajikan dalam bentuk tabel.

```
# Membuat koneksi ke database
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()

# Menjalankan query untuk menemukan nasabah dengan nama 'Agus Santoso'
query = '''
SELECT * FROM Nasabah
WHERE Nama_Nasabah = 'Agus Santoso'
'''

# Membaca hasil query ke dalam DataFrame
dfs2 = pd.read_sql_query(query, conn)

# Menutup koneksi
conn.close()
dfs2
```

- Selanjutnya, kami mencoba untuk menyeleksi sekaligus menampilkan jenis transaksi yang bersifat debet atau `T0` dengan rincian sebagai berikut. Mengambil data dari tabel "Transaksi" dengan melakukan join dengan tabel "Jenis_Transaksi" berdasarkan relasi "ID_Jenis_Transaksi". Pada query ini, terdapat empat kolom yang diambil, yaitu "ID_Transaksi", "Tanggal_Transaksi", "Nominal_Transaksi" dari tabel "Transaksi", serta "Jenis_Transaksi" dari tabel "Jenis_Transaksi". Pengambilan data dibatasi oleh kondisi yang menyatakan bahwa "ID_Jenis_Transaksi" harus bernilai 'T0'.

```
# Membuat koneksi ke database
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()

# Menjalankan query untuk menemukan transaksi dengan jenis transaksi 'T1'
query = '''
SELECT Transaksi.ID_Transaksi, Transaksi.Tanggal_Transaksi, Transaksi.Nominal_Transaksi, Jenis_Transaksi.Jenis_Transaksi
FROM Transaksi
JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
WHERE Jenis_Transaksi.ID_Jenis_Transaksi = 'T0'
'''

# Membaca hasil query ke dalam DataFrame
dfs1 = pd.read_sql_query(query, conn)

# Menutup koneksi
conn.close()

# Menampilkan DataFrame sebagai tabel
print("Data dengan Jenis Transaksi T0:")
dfs1
```

- Terakhir, kami melakukan implementasi fungsi *select* dan *order by,* yaitu memilih tabel nasabah dan dilakukan pengurutan berdasarkan tahun lahir dengan rincian sebagai berikut. Menjalankan query SQL untuk memilih semua data dari tabel "Nasabah" dan mengurutkannya berdasarkan tahun lahir yang diekstrak dari kolom "TTL_Nasabah" menggunakan substr(TTL_Nasabah, 7, 4), yang mengambil empat karakter mulai dari posisi ketujuh dalam format 'DD-MM-YYYY'. Hasil dari query tersebut dibaca ke dalam sebuah DataFrame menggunakan pandas dan koneksi ke database ditutup setelah query

dijalankan dan akan menampilkan data nasabah yang sudah diurutkan berdasarkan tahun lahir.

```
# Membuat koneksi ke database
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()

# Menjalankan query untuk memilih semua nasabah dan mengurutkan berdasarkan TTL_Nasabah
query = '''
SELECT * FROM Nasabah
ORDER BY substr(TTL_Nasabah, 7, 4)
'''

# Membaca hasil query ke dalam DataFrame
dfs3 = pd.read_sql_query(query, conn)

# Menutup koneksi
conn.close()

# Menampilkan DataFrame sebagai tabel
print("Nasabah yang diurutkan berdasarkan tahun lahir")
dfs3
```

4. Fungsi Join

Pada implementasi fungsi *join* ini, kami mencoba untuk mengkombinasikan fungsi *join* dan fungsi *select.* Berikut rinciannya.

a. Menggabungkan fungsi join dan select untuk mengambil data nasabah dan data nominal transaksi serta menggabungkannya dengan kriteria nominal transaksi diatas 150.000

```
# Menjalankan query untuk mengambil/SELECT data dengan JOIN antara tabel
query = """
SELECT Nasabah.Nama_Nasabah, Rekening_Bank.ID_Akun, Nasabah.ID_Nasabah, Transaksi.
FROM Transaksi
JOIN Rekening_Bank ON Transaksi.ID_Akun = Rekening_Bank.ID_Akun
JOIN Nasabah ON Rekening_Bank.ID_Nasabah = Nasabah.ID_Nasabah
WHERE Transaksi.Nominal_Transaksi > 150000;
"""
c.execute(query)
```

b. Menggunakan fungsi *select* untuk mengambil data nasabah dan nominal transaksinya dan menggabungkannya dengan membuat terlebih dahulu maximal transaksi dari tiap nasabah dan hasilnya akan digabungkan.

```
# Menjalankan query untuk mengambil data dengan max(Nominal_Transaksi) dari setiap Nama_Nasabah
query = """
WITH MaxTransaksi AS (
    SELECT Rekening_Bank.ID_Nasabah, MAX(Transaksi.Nominal_Transaksi) AS Max_Nominal_Transaksi
    FROM Transaksi
    JOIN Rekening_Bank ON Transaksi.ID_Akun = Rekening_Bank.ID_Akun
    GROUP BY Rekening_Bank.ID_Nasabah
)
SELECT Nasabah.Nama_Nasabah, Rekening_Bank.ID_Akun, Nasabah.ID_Nasabah, Transaksi.ID_Transaksi, Trans
FROM MaxTransaksi
JOIN Rekening_Bank ON MaxTransaksi.ID_Nasabah = Rekening_Bank.ID_Nasabah
JOIN Transaksi ON Rekening_Bank.ID_Akun = Transaksi.ID_Akun AND
MaxTransaksi.Max_Nominal_Transaksi = Transaksi.Nominal_Transaksi
JOIN Nasabah ON Rekening_Bank.ID_Nasabah = Nasabah.ID_Nasabah;
"""
```

3.2 Implementasi Interaktif (GUI) dan *Testing*

Pada subbab implementasi GUI ini, akan dijelaskan proses implementasi basis data menggunakan *Structured Query Language* (SQL) secara interaktif, khususnya implementasi yang bersesuaian dengan tujuan awal *mini project* ini.

1. Memasukkan Data Transaksi Baru ke Basis Data

   Pada implementasi ini, akan dilakukan pembuatan *widget input* dan *insert* data ke tabel Transaksi. Berikut penjelasan lebih lanjut.

   - Membuat *widget input* ID Transaksi, ID Akun, ID Jenis Transaksi, Tanggal Transaksi, Nominal Transaksi, ID Biaya Admin, Tombol Submit.

```
# Membuat widget input
id_transaksi_input = widgets.Text(
    value='',
    description='ID Trans:',
    disabled=False)

id_akun_input = widgets.Text(
    value='',
    description='ID Akun:',
    disabled=False)

id_jenis_transaksi_input = widgets.Dropdown(
    options=['T0', 'T1', 'T2', 'T3', 'T4'],
    value='T0',
    description='ID Jenis Trans:',
    disabled=False)

tanggal_transaksi_input = widgets.DatePicker(
    description='Tgl Trans:',
    disabled=False,
    value=datetime.now())
```

```
nominal_transaksi_input = widgets.IntText(
    value=0,
    description='Nominal:',
    disabled=False)

id_biaya_admin_input = widgets.Text(
    value='',
    description='Biaya Admin:',
    disabled=False)

submit_button = widgets.Button(
    description='Masukkan Data',
    disabled=False,
    button_style='success',
    tooltip='Klik untuk memasukkan data transaksi')
```

   - Membuat fungsi untuk *insert* data

```
def insert_data(b):
    with sqlite3.connect('perbankan.db') as conn:
        c = conn.cursor()
        # Mengaktifkan foreign key
        c.execute('PRAGMA foreign_keys = ON')
        try:
            c.execute('''INSERT INTO Transaksi (ID_Transaksi, ID_Akun, ID_Jenis_Transaksi,
                         Tanggal_Transaksi, Nominal_Transaksi, ID_Biaya_Admin)
                         VALUES (?, ?, ?, ?, ?, ?)''',
                      (id_transaksi_input.value, id_akun_input.value,
                       id_jenis_transaksi_input.value, tanggal_transaksi_input.value.
                       strftime('%d-%m-%Y'), nominal_transaksi_input.value,
                       id_biaya_admin_input.value))
            print("Data transaksi berhasil dimasukkan.")
        except sqlite3.IntegrityError as e:
            print("Gagal memasukkan data: ", e)
        except Exception as e:
            print("Terjadi kesalahan: ", e)
```

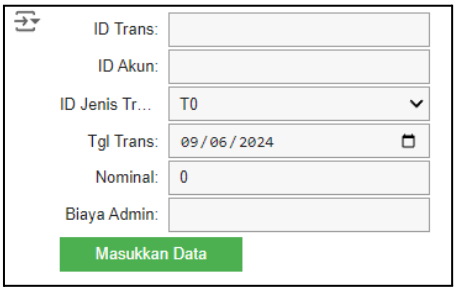
- Menghubungkan tombol dengan fungsi

```
submit_button.on_click(insert_data)
```

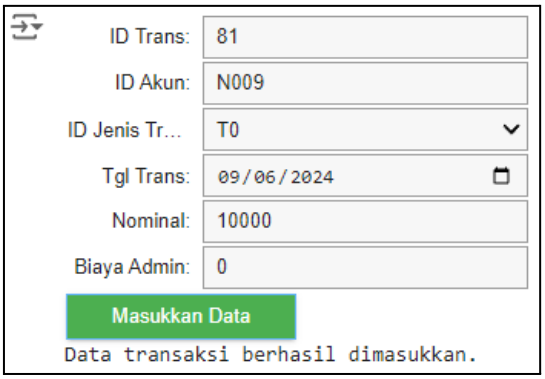
- Menampilkan *widget*

```
display(id_transaksi_input, id_akun_input, id_jenis_transaksi_input, tanggal_transaksi_input,
        nominal_transaksi_input, id_biaya_admin_input, submit_button)
```

Setelah *code* di atas sudah dijalankan, akan muncul *user interface* sebagai berikut.



Sebagai simulasi, akan coba diisi ID Transaksi, ID Akun, ID Jenis Transaksi, Tanggal Transaksi, Nominal Transaksi, Biaya Admin di atas, lalu di-*klik* tombol Masukkan Data untuk memasukkan data tersebut.



Kemudian akan dicetak kalimat “Data transaksi berhasil dimasukkan..” apabila tidak ada kesalahan input, seperti duplikasi ID Transaksi.

2. Cek Mutasi Berdasarkan Tanggal Transaksi

Pada implementasi cek mutasi berdasarkan rentang tanggal transaksi ini, akan dilakukan pembuatan *widget input,* validasi *login,* serta pengecekan dan penampilan hasil mutasi. Berikut penjelasan lebih lanjut.

- Membuat *widget* untuk *input* Username dan Password.

```
# Widget untuk input username dan password
username_input = widgets.Text(
    value='',
    description='Username:',
    style={'description_width': 'initial'})

password_input = widgets.Password(
    value='',
    description='Password:',
    style={'description_width': 'initial'})

login_button = widgets.Button(
    description='Login',
    button_style='success',
    tooltip='Klik untuk login')

output_login = widgets.Output()
```

  - `username_input` : *widget* untuk *input username*.
  - `password_input` : widget untuk *input password*.
  - `login_button` : tombol untuk melakukan *login*.
  - `output_login` : output widget untuk menampilkan hasil *login*.

- Membuat *widget* untuk *input* Tanggal Awal dan Akhir

```
# Widget untuk input tanggal awal dan akhir
tanggal_awal_widget = widgets.DatePicker(
    description='Tanggal Awal:',
    style={'description_width': 'initial'},
    layout={'visibility': 'hidden'})  # Awalnya tidak terlihat

tanggal_akhir_widget = widgets.DatePicker(
    description='Tanggal Akhir:',
    style={'description_width': 'initial'},
    layout={'visibility': 'hidden'})  # Awalnya tidak terlihat

cek_mutasi_button = widgets.Button(
    description='Cek Mutasi',
    button_style='info',
    tooltip='Cek mutasi berdasarkan rentang tanggal',
    layout={'visibility': 'hidden'})  # Awalnya tidak terlihat

output_mutasi = widgets.Output()
```

  - `tanggal_awal_widget` : widget untuk memilih tanggal awal.
  - `tanggal_akhir_widget` : widget untuk memilih tanggal akhir.
  - `cek_mutasi_button` : tombol untuk mengecek mutasi transaksi.
  - `output_mutasi` : output widget untuk menampilkan hasil mutasi transaksi.

- Membuat fungsi validasi *user*

```
def validasi_user(username, password):
    conn = sqlite3.connect('perbankan.db')
    c = conn.cursor()
    c.execute('SELECT COUNT(*) FROM Akun_MBanking WHERE Username = ? AND Password = ?', (username, password))
    hasil = c.fetchone()[0]
    conn.close()
    return hasil > 0
```

Fungsi `validasi_user` memeriksa apakah kombinasi `username` dan `password` yang dimasukkan ada dalam tabel `Akun_MBanking` dengan menggunakan query SQL `SELECT COUNT(*)` untuk menghitung jumlah baris yang sesuai dengan kondisi yang diberikan, `FROM Akun_MBanking` untuk menunjukkan bahwa pencarian dilakukan pada tabel `Akun_MBanking`, serta `WHERE Username = ? AND Password = ?` adalah kondiis untuk mencocokkan baris yang memiliki `Username` dan ` Password` sesuai dengan nilai yang diberikan. Jika ada baris yang sesuai (hasil > 0), maka user tervalidasi atau berhasil login.

- Membuat fungsi menangani *login*

```
# Fungsi untuk menangani login
def login(b):
    with output_login:
        clear_output()
        username = username_input.value
        password = password_input.value

        if validasi_user(username, password):
            print("Login berhasil. Silakan pilih rentang tanggal untuk cek mutasi.")
            tanggal_awal_widget.layout.visibility = 'visible'
            tanggal_akhir_widget.layout.visibility = 'visible'
            cek_mutasi_button.layout.visibility = 'visible'
        else:
            print("Username atau Password Anda salah.")
```

Fungsi `login` menangani proses *login*. Jika validasi akun berhasil, *widget input* tanggal dan tombol cek mutasi akan ditampilkan sehingga user yang sudah tervalidasi dapat memilih rentang tanggal transaksi yang ingin dicek mutasinya.

- Membuat fungsi mengecek mutasi

```
def cek_mutasi(b):
    with output_mutasi:
        clear_output()
        username = username_input.value
        password = password_input.value
        tanggal_awal = tanggal_awal_widget.value
        tanggal_akhir = tanggal_akhir_widget.value

        if not tanggal_awal or not tanggal_akhir:
            print("Harap pilih rentang tanggal yang valid.")
            return

        conn = sqlite3.connect('perbankan.db')
        c = conn.cursor()
        c.execute('''SELECT Transaksi.ID_Transaksi,
                            Transaksi.Tanggal_Transaksi,
                            Transaksi.Nominal_Transaksi,
                            Jenis_Transaksi.Jenis_Transaksi,
                            Biaya_Admin.Nominal_Biaya_Admin
                     FROM Transaksi
                     JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
                     JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
                     WHERE Transaksi.ID_Akun = (SELECT ID_Akun FROM Akun_MBanking WHERE Username = ?)
                     AND Transaksi.Tanggal_Transaksi BETWEEN ? AND ?''',
                  (username, tanggal_awal.strftime('%d-%m-%Y'), tanggal_akhir.strftime('%d-%m-%Y')))

        mutasi = c.fetchall()
        conn.close()
```

```
        if mutasi:
            df = pd.DataFrame(mutasi, columns=['ID Transaksi', 'Tanggal Transaksi',
                                               'Nominal Transaksi', 'Jenis Transaksi', 'Biaya Admin'])
            display(df)

            # Menghitung total debit dan kredit
            total_debit = df[df['Jenis Transaksi'] == 'Debit']['Nominal Transaksi'].sum()
            total_kredit = df[df['Jenis Transaksi'] != 'Debit']['Nominal Transaksi'].sum()
            selisih = total_debit - total_kredit

            # Menampilkan total debit dan kredit
            print(f"Total Debit: {total_debit}")
            print(f"Total Kredit: {total_kredit}")
            print(f"Selisih: {selisih}")
        else:
            print("Tidak ada transaksi dalam rentang tanggal tersebut.")
```

Fungsi `cek_mutasi` mengambil data transaksi berdasarkan rentang tanggal yang dipilih dan menampilkan hasilnya. Fungsi ini juga melibatkan query SQL, sebagai berikut.

- Mengambil kolom `ID_Transaksi`, `Tanggal_Transaksi`, `Nominal_Transaksi`, `Jenis_Transaksi`, dan `Nominal_Biaya_Admin` dari tabel `Transaksi`, `Jenis_Transaksi`, dan `Biaya_Admin`.

```
'''SELECT Transaksi.ID_Transaksi,
          Transaksi.Tanggal_Transaksi,
          Transaksi.Nominal_Transaksi,
          Jenis_Transaksi.Jenis_Transaksi,
          Biaya_Admin.Nominal_Biaya_Admin
```

- Menunjukkan bahwa data diambil dari tabel `Transaksi` (menjadi tabel utama).

```
'''SELECT Transaksi.ID_Transaksi,
        Transaksi.Tanggal_Transaksi,
        Transaksi.Nominal_Transaksi,
        Jenis_Transaksi.Jenis_Transaksi,
        Biaya_Admin.Nominal_Biaya_Admin
 FROM Transaksi
```

- Menggabungkan tabel `Transaksi` dengan tabel `Biaya_Admin` berdasarkan kolom `ID_Biaya_Admin`.

```
'''SELECT Transaksi.ID_Transaksi,
        Transaksi.Tanggal_Transaksi,
        Transaksi.Nominal_Transaksi,
        Jenis_Transaksi.Jenis_Transaksi,
        Biaya_Admin.Nominal_Biaya_Admin
 FROM Transaksi
 JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
```

- Menggabungkan tabel `Transaksi` dengan tabel `Jenis_Transaksi` berdasarkan kolom `ID_Jenis_Transaksi`.

```
'''SELECT Transaksi.ID_Transaksi,
        Transaksi.Tanggal_Transaksi,
        Transaksi.Nominal_Transaksi,
        Jenis_Transaksi.Jenis_Transaksi,
        Biaya_Admin.Nominal_Biaya_Admin
 FROM Transaksi
 JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
 JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
```

- Menentukan kondisi bahwa `ID_Akun` dari tabel `Transaksi` harus sesuai dengan `ID_Akun` yang didapat dari tabel `Akun_MBanking` berdasarkan `Username` yang diberikan sebagai parameter.

```
'''SELECT Transaksi.ID_Transaksi,
        Transaksi.Tanggal_Transaksi,
        Transaksi.Nominal_Transaksi,
        Jenis_Transaksi.Jenis_Transaksi,
        Biaya_Admin.Nominal_Biaya_Admin
 FROM Transaksi
 JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
 JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
 WHERE Transaksi.ID_Akun = (SELECT ID_Akun FROM Akun_MBanking WHERE Username = ?)
```

- Menentukan kondisi bahwa `Tanggal_Transaksi` harus berada di antara `tanggal_awal` dan `tanggal_akhir` yang diberikan sebagai parameter. Dengan format `tanggal_awal` dan `tanggal_akhir` adalah `%d-%m-%Y`.

```
'''SELECT Transaksi.ID_Transaksi,
        Transaksi.Tanggal_Transaksi,
        Transaksi.Nominal_Transaksi,
        Jenis_Transaksi.Jenis_Transaksi,
        Biaya_Admin.Nominal_Biaya_Admin
 FROM Transaksi
 JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
 JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
 WHERE Transaksi.ID_Akun = (SELECT ID_Akun FROM Akun_MBanking WHERE Username = ?)
 AND Transaksi.Tanggal_Transaksi BETWEEN ? AND ?''',
(username, tanggal_awal.strftime('%d-%m-%Y'), tanggal_akhir.strftime('%d-%m-%Y')))
```

- Menghubungkan tombol dengan fungsi, di mana `login_button.on_click(login)` untuk menghubungkan tombol login dengan fungsi `login`, `cek_mutasi_button.on_click(cek_mutasi)` untuk menghubungkan tombol cek mutasi dengan fungsi `cek_mutasi`, dan `display()` untuk menampilkan semua widget pada *user interface*.

```
login_button.on_click(login)
cek_mutasi_button.on_click(cek_mutasi)

display(username_input, password_input, login_button, output_login, tanggal_awal_widget,
        tanggal_akhir_widget, cek_mutasi_button, output_mutasi)
```

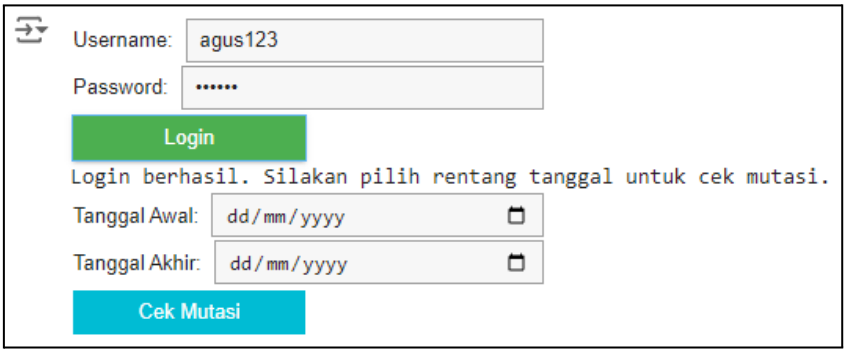
Setelah *code* di atas sudah dijalankan, akan muncul *user interface* sebagai berikut.

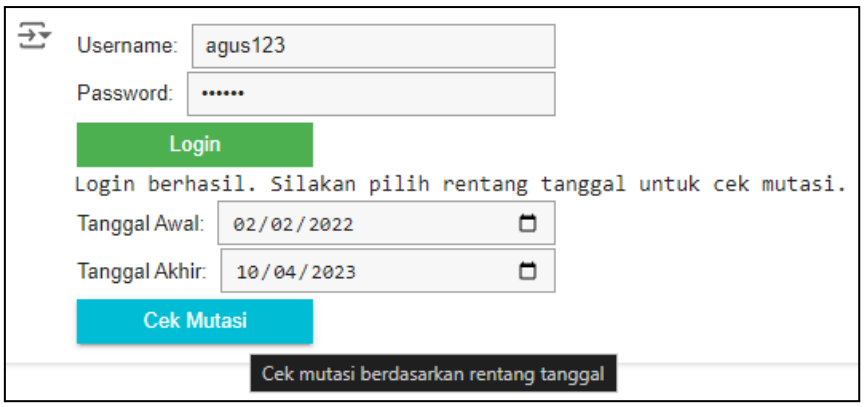


Sebagai simulasi, akan coba diisi Username dan Password di atas, lalu di-*klik* tombol Login untuk mengecek validasi username dan password tersebut.

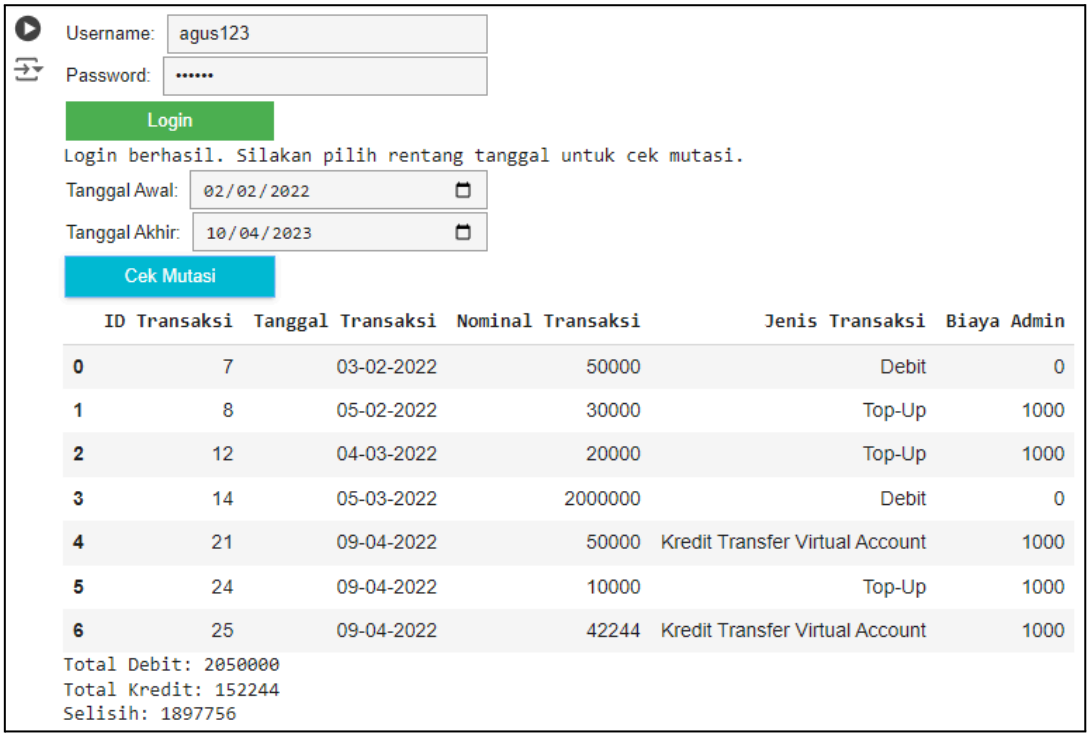


Kemudian akan dicetak kalimat “Login berhasil. Silahkan pilih rentang tanggal untuk cek mutasi.” apabila Username dan Password benar. Pengguna akan diarahkan untuk memilih tanggal awal dan tanggal akhir untuk cek mutas, seperti berikut.





Setelah pengguna klik tombol `Cek Mutasi`. akan ditampilkan tabel mutasi berdasarkan tanggal lengkap dengan total kredit, total debit, dan selisih nominal transaksi dalam rentang waktu yang dipilih, seperti berikut.



	ID Transaksi	Tanggal Transaksi	Nominal Transaksi	Jenis Transaksi	Biaya Admin
0	7	03-02-2022	50000	Debit	0
1	8	05-02-2022	30000	Top-Up	1000
2	12	04-03-2022	20000	Top-Up	1000
3	14	05-03-2022	2000000	Debit	0
4	21	09-04-2022	50000	Kredit Transfer Virtual Account	1000
5	24	09-04-2022	10000	Top-Up	1000
6	25	09-04-2022	42244	Kredit Transfer Virtual Account	1000

Total Debit: 2050000
Total Kredit: 152244
Selisih: 1897756

3. Cek Mutasi Berdasarkan Jenis dan Tanggal Transaksi

Pada implementasi cek mutasi berdasarkan jenis dan rentang tanggal transaksi ini, akan dilakukan pembuatan *widget input,* validasi *login,* serta pengecekan dan penampilan hasil mutasi berdasarkan jenis sekaligus rentang tanggal waktu transaksi. Berikut penjelasan lebih lanjut.

- Membuat *widget* untuk *input* Username dan Password.

```
# Widget input username dan password
username_input = widgets.Text(
    value='',
    description='Username:',
    style={'description_width': 'initial'})

password_input = widgets.Password(
    value='',
    description='Password:',
    style={'description_width': 'initial'})

login_button = widgets.Button(
    description='Login',
    button_style='success',
    tooltip='Klik untuk login')

output_login = widgets.Output()

login_button.on_click(login)

display(username_input, password_input, login_button, output_login)
```

- `username_input` : *widget* untuk *input username*.
- `password_input` : widget untuk *input password*.
- `login_button` : tombol untuk melakukan *login*.
- `output_login` : output widget untuk menampilkan hasil *login*.

- Membuat *widget* untuk memilih jenis transaksi

```
# Widget untuk memilih jenis transaksi dengan label deskriptif
jenis_transaksi_dropdown = widgets.Dropdown(
    options=[
        ('Debit', 'T0'),
        ('Kredit Transfer Antar Rekening', 'T1'),
        ('Kredit Transfer Antar Bank', 'T2'),
        ('Kredit Transfer Virtual Account', 'T3'),
        ('Top-Up', 'T4')
    ],
    value='T0',
    description='Jenis Transaksi:',
    style={'description_width': 'initial'},
    layout={'visibility': 'hidden'})  # Awalnya widget ini tidak terlihat
```

- `Jenis Transaksi` : *widget* untuk input jenis transaksi yang dipilih
- `T0` : Debit
- `T1` : Kredit transfer antar rekening
- `T2` : Kredit transfer antar bank
- `T3` : Kredit transfer *virtual account*

- Membuat *widget* untuk memilih rentang waktu tanggal transaksi

```
# Widget untuk memilih tanggal awal dan akhir
tanggal_awal_picker = widgets.DatePicker(
    description='Tanggal Awal:',
    style={'description_width': 'initial'},
    layout={'visibility': 'hidden'})  # Awalnya widget ini tidak terlihat

tanggal_akhir_picker = widgets.DatePicker(
    description='Tanggal Akhir:',
    style={'description_width': 'initial'},
    layout={'visibility': 'hidden'})  # Awalnya widget ini tidak terlihat

# Tombol submit
submit_button = widgets.Button(
    description='Submit',
    button_style='info',
    tooltip='Klik untuk melihat mutasi transaksi',
    layout={'visibility': 'hidden'})  # Awalnya tombol ini tidak terlihat

output_mutasi = widgets.Output()
```

- `tanggal_awal_widget` : widget untuk memilih tanggal awal.
- `tanggal_akhir_widget` : widget untuk memilih tanggal akhir.
- ``login_button` : tombol untuk melakukan *login*.
- `output_login` : output widget untuk menampilkan hasil *login*.

- Membuat fungsi validasi *user*

```
# Fungsi untuk validasi user
def validasi_user(username, password):
    conn = sqlite3.connect('perbankan.db')
    c = conn.cursor()
    c.execute('SELECT * FROM Akun_MBanking WHERE Username = ? AND Password = ?', (username, password))
    result = c.fetchone()
    conn.close()
    return result is not None
```

Fungsi validasi_user bertujuan untuk memvalidasi pengguna berdasarkan username dan password yang diberikan. Dalam fungsinya, pertama-tama dibuat koneksi ke database SQLite bernama 'perbankan.db'. Selanjutnya, sebuah cursor digunakan untuk menjalankan query SQL yang memilih semua kolom dari tabel "Akun_MBanking" di mana nilai "Username" dan "Password" sesuai dengan parameter yang diberikan. Fungsi menggunakan parameterized query untuk mencegah SQL injection. Setelah query dieksekusi, hasilnya diambil menggunakan fetchone(), yang akan mengembalikan baris pertama dari hasil query atau None jika tidak ada baris yang cocok. Koneksi ke database kemudian ditutup, dan fungsi mengembalikan True jika hasilnya tidak None, menandakan bahwa username dan password valid, atau False jika tidak valid.

- Membuat fungsi menangani *login*

```
# Fungsi untuk menangani login
def login(b):
    with output_login:
        clear_output()
        username = username_input.value
        password = password_input.value

        if validasi_user(username, password):
            print("Login berhasil. Silakan pilih jenis transaksi dan rentang tanggal.")
            jenis_transaksi_dropdown.layout.visibility = 'visible'
            tanggal_awal_picker.layout.visibility = 'visible'
            tanggal_akhir_picker.layout.visibility = 'visible'
            submit_button.layout.visibility = 'visible'
        else:
            print("Username atau Password Anda salah.")
```

Fungsi `login` menangani proses *login*. Jika validasi akun berhasil, *widget input* tanggal, tombol jenis transaksi, dan tombol *submit* akan ditampilkan sehingga user yang sudah tervalidasi dapat memilih rentang tanggal transaksi dan jenis transaksi yang akan dicek mutasinya. Jika validasi akun salah, maka akan menampilkan *username* atau *password* anda salah.

- Membuat fungsi mengecek mutasi

```
def submit(b):
    with output_mutasi:
        clear_output()
        jenis_transaksi = jenis_transaksi_dropdown.value
        tanggal_awal = tanggal_awal_picker.value
        tanggal_akhir = tanggal_akhir_picker.value

        if not tanggal_awal or not tanggal_akhir:
            print("Harap pilih rentang tanggal yang valid.")
            return

        conn = sqlite3.connect('perbankan.db')
        c = conn.cursor()
        query = '''SELECT Transaksi.ID_Transaksi,
                          Transaksi.Tanggal_Transaksi,
                          Transaksi.Nominal_Transaksi,
                          Jenis_Transaksi.Jenis_Transaksi,
                          Biaya_Admin.Nominal_Biaya_Admin
                   FROM Transaksi
                   JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
                   JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
                   WHERE Transaksi.ID_Akun = (SELECT ID_Akun FROM Akun_MBanking WHERE Username = ?)
                   AND Transaksi.Tanggal_Transaksi BETWEEN ? AND ?
                   AND Jenis_Transaksi.ID_Jenis_Transaksi = ?'''
        params = (username_input.value, tanggal_awal.strftime('%d-%m-%Y'), tanggal_akhir.strftime('%d-%m-%Y'), jenis_transaksi)
        c.execute(query, params)
```

```
if mutasi:
    # Membuat DataFrame dari hasil query
    df = pd.DataFrame(mutasi, columns=['ID Transaksi', 'Tanggal Transaksi'
    , 'Nominal Transaksi', 'Jenis Transaksi', 'Biaya Admin'])
    display(df)
    total_debit = 0
    total_kredit = 0

    for index, row in df.iterrows():
        jenis_mutasi = 'Debit' if row['Jenis Transaksi'] in ['Debit'] else 'Kredit'
        if jenis_mutasi == 'Debit':
            total_debit += row['Nominal Transaksi']
        else:
            total_kredit += row['Nominal Transaksi']
    selisih = total_debit - total_kredit

    # Menampilkan total debit dan kredit
    print(f"Total Debit: {total_debit}")
    print(f"Total Kredit: {total_kredit}")
    print(f"Selisih: {selisih}")

else:
    print("Tidak ada transaksi dalam rentang tanggal tersebut.")
```

Fungsi `cek_mutasi` mengambil data transaksi berdasarkan rentang tanggal yang dipilih dan menampilkan hasilnya. Fungsi ini juga melibatkan query SQL, sebagai berikut.

```
conn = sqlite3.connect('perbankan.db')
c = conn.cursor()
query = '''SELECT Transaksi.ID_Transaksi,
                  Transaksi.Tanggal_Transaksi,
                  Transaksi.Nominal_Transaksi,
                  Jenis_Transaksi.Jenis_Transaksi,
                  Biaya_Admin.Nominal_Biaya_Admin
            FROM Transaksi
            JOIN Biaya_Admin ON Transaksi.ID_Biaya_Admin = Biaya_Admin.ID_Biaya_Admin
            JOIN Jenis_Transaksi ON Transaksi.ID_Jenis_Transaksi = Jenis_Transaksi.ID_Jenis_Transaksi
            WHERE Transaksi.ID_Akun = (SELECT ID_Akun FROM Akun_MBanking WHERE Username = ?)
            AND Transaksi.Tanggal_Transaksi BETWEEN ? AND ?
            AND Jenis_Transaksi.ID_Jenis_Transaksi = ?'''
params = (username_input.value, tanggal_awal.strftime('%d-%m-%Y'),
          tanggal_akhir.strftime('%d-%m-%Y'), jenis_transaksi)
c.execute(query, params)
```

- Bagian `SELECT` memilih kolom-kolom yang diinginkan dari tabel "Transaksi", "Jenis_Transaksi", dan "Biaya_Admin". Kolom-kolom yang dipilih adalah "ID_Transaksi", "Tanggal_Transaksi", "Nominal_Transaksi" dari tabel "Transaksi", "Jenis_Transaksi" dari tabel "Jenis_Transaksi", dan "Nominal_Biaya_Admin" dari tabel "Biaya_Admin".
- Bagian `JOIN` melakukan join antara tabel "Transaksi" dengan tabel "Biaya_Admin" berdasarkan kolom "ID_Biaya_Admin", serta join antara tabel "Transaksi" dengan tabel "Jenis_Transaksi" berdasarkan kolom "ID_Jenis_Transaksi".
- Bagian ini menetapkan beberapa kondisi:
    1. Kondisi pertama menggunakan subquery untuk mencari "ID_Akun" dari tabel "Akun_MBanking" yang sesuai dengan "Username" yang diberikan.
    2. Kondisi kedua membatasi hasil berdasarkan rentang tanggal transaksi yang ditentukan oleh dua parameter: tanggal awal dan tanggal akhir.
    3. Kondisi ketiga membatasi hasil berdasarkan "ID_Jenis_Transaksi" yang sesuai dengan parameter yang diberikan.

- Pada bagian `params`, disiapkan dengan nilai-nilai yang diperlukan untuk eksekusi query, termasuk username_input.value yang mengambil username dari input pengguna, `tanggal_awal.strftime` dan `tanggal_akhir.strftime` yang mengonversi objek tanggal tanggal_awal dan tanggal_akhir ke format string 'DD-MM-YYYY', serta jenis_transaksi yang merupakan nilai dari jenis transaksi yang dicari.

```
if mutasi:
    # Membuat DataFrame dari hasil query
    df = pd.DataFrame(mutasi, columns=['ID Transaksi', 'Tanggal Transaksi'
    , 'Nominal Transaksi', 'Jenis Transaksi', 'Biaya Admin'])
    display(df)
    total_debit = 0
    total_kredit = 0

    for index, row in df.iterrows():
        jenis_mutasi = 'Debit' if row['Jenis Transaksi'] in ['Debit'] else 'Kredit'
        if jenis_mutasi == 'Debit':
            total_debit += row['Nominal Transaksi']
        else:
            total_kredit += row['Nominal Transaksi']
    selisih = total_debit - total_kredit

    # Menampilkan total debit dan kredit
    print(f"Total Debit: {total_debit}")
    print(f"Total Kredit: {total_kredit}")
    print(f"Selisih: {selisih}")

else:
    print("Tidak ada transaksi dalam rentang tanggal tersebut.")
```

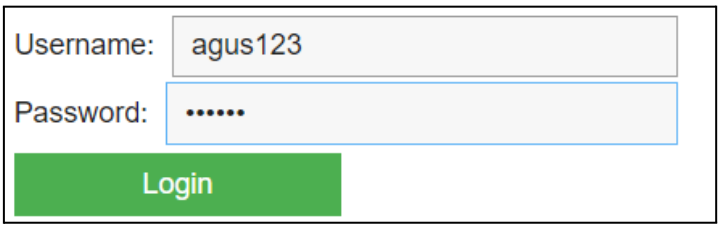
- Kode ini memeriksa apakah terdapat data transaksi dalam variabel mutasi. Jika ada, data tersebut diubah menjadi DataFrame dengan kolom 'ID Transaksi', 'Tanggal Transaksi', 'Nominal Transaksi', 'Jenis Transaksi', dan 'Biaya Admin', lalu ditampilkan. Dua variabel, total_debit dan total_kredit, diinisialisasi untuk menghitung total debit dan kredit. Setiap baris dalam DataFrame diiterasi untuk menentukan jenis mutasi ('Debit' atau 'Kredit') berdasarkan 'Jenis Transaksi'.
- Dua variabel, total_debit dan total_kredit, diinisialisasi untuk menghitung total debit dan kredit. Setiap baris dalam DataFrame diiterasi untuk menentukan jenis mutasi ('Debit' atau 'Kredit') berdasarkan 'Jenis Transaksi'.
- Nilai 'Nominal Transaksi' ditambahkan ke total_debit jika jenisnya 'Debit', atau ke total_kredit jika sebaliknya. Setelah iterasi, selisih antara total debit dan kredit dihitung dan ditampilkan bersama total debit dan kredit. Jika tidak ada transaksi, pesan "Tidak ada transaksi dalam rentang tanggal tersebut." akan ditampilkan.

- Menghubungkan tombol dengan fungsi, di mana `login_button.on_click(login)` untuk menghubungkan tombol login dengan fungsi `login`, `cek_mutasi_button.on_click(cek_mutasi)` untuk menghubungkan tombol cek mutasi dengan fungsi `cek_mutasi`, dan `display()` untuk menampilkan semua widget pada *user interface*.

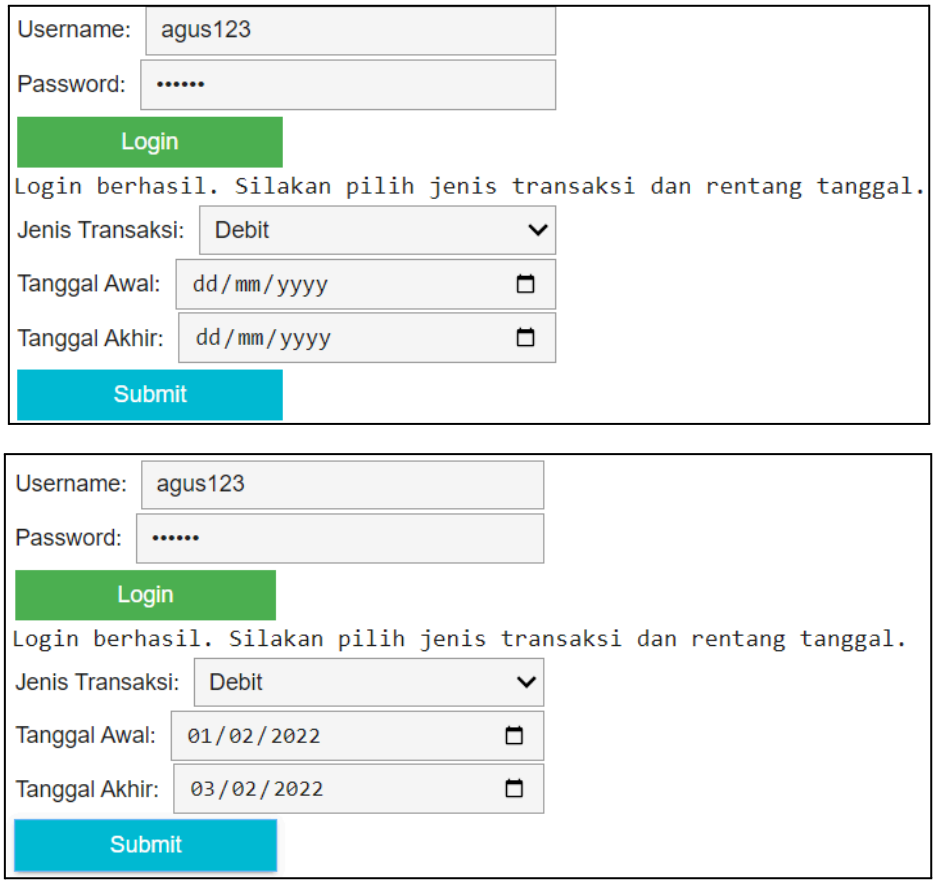
```
login_button.on_click(login)
submit_button.on_click(submit)

display(jenis_transaksi_dropdown, tanggal_awal_picker, tanggal_akhir_picker, submit_button, output_mutasi)
```

Ketika *code* di atas dijalankan, akan muncul *user interface* sebagai berikut.



Sebagai simulasi, akan dilakukan pengujian *log-in* dengan memasukkan input *username* dan *password* dari salah satu nasabah, setelah itu di-*klik* tombol Login untuk mengecek validasi username dan password tersebut.



Ketika *log-in* sudah berhasil, selanjutnya *code* akan menampilkan *widget* untuk memilih jenis transaksi dan rentang waktu tanggal yang akan dicek mutasinya. *User* bisa secara bebas memilih jenis transaksi dan rentang waktu tanggal yang ingin dipilih seperti pada contoh gambar di atas. Setelah mengetuk *widget submit*, maka *output* akan menampilkan tabel mutasi berdasarkan id transaksi, tanggal transaksi, nominal transaksi, jenis transaksi, dan biaya admin sesuai pada gambar di bawah.

Username: agus123

Password: ••••••

Login

Login berhasil. Silakan pilih jenis transaksi dan rentang tanggal.

Jenis Transaksi: Debit

Tanggal Awal: 01/02/2022

Tanggal Akhir: 27/04/2022

Submit

	ID Transaksi	Tanggal Transaksi	Nominal Transaksi	Jenis Transaksi	Biaya Admin
0	7	03-02-2022	50000	Debit	0
1	14	05-03-2022	2000000	Debit	0
2	18	25-03-2022	1000000	Debit	0

Total Debit: 3050000
Total Kredit: 0
Selisih: 3050000

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan penjabaran laporan *mini project* di atas, dapat disimpulkan bahwa implementasi sistem basis data untuk layanan *mobile banking* yang dirancang telah berhasil memenuhi kebutuhan operasional bank secara sederhana. Sistem basis data yang dirancang mampu menyediakan akses cepat dan andal terhadap informasi seputar transaksi yang dilakukan nasabah. Implementasi sistem basis data yang dilakukan juga dapat mengelola banyak transaksi harian yang dilakukan oleh nasabah dengan cepat dan akurat.

Sistem basis data ini juga menyediakan mekanisme yang sederhana dan efektif untuk menganalisis data dengan cepat dan menghasilkan laporan transaksi berupa mutasi yang informatif. Hal ini memudahkan nasabah untuk dapat memverifikasi serta memantau aktivitas finansial secara *real-time.* Adanya implementasi SQL dan GUI berhasil memfasilitasi pengguna dalam memasukkan dan mengecek data transaksi dengan mudah.

4.2 Saran

Untuk pengembangan lebih lanjut, *mini project* ini dapat dikembangkan dengan menginput atribut saldo awal atau tabungan awal sehingga basis data dapat mencakup saldo untuk tiap transaksi yang dilakukan untuk optimasi *mobile banking* ke depannya.

DAFTAR PUSTAKA

Mukhlis, I. R., Hermansyah, D., & Lantang, V. M. (2023, May 1). *Rancangan Basis Data Transaksi Pada PT.Bank Perkreditan Rakyat ABC Menggunakan MySQL Dengan Model Entity Relationship Diagram (ERD) dan*. E-Journal Institut Teknologi Telkom Surabaya. Retrieved June 9, 2024, from https://journal.ittelkom-sby.ac.id/jaiit/article/download/305/204

Putra, A. M. (2019). *SISTEM BASIS DATA DALAM BANK by Aditya Marta Putra*. Prezi. Retrieved June 9, 2024, from https://prezi.com/p/8_pacrswnn4r/sistem-basis-data-dalam-bank/

LAMPIRAN

SQL & GUI Final Project Database Kelompok 6

Data-Perbankan